

DIFFERENTIABLE FORWARD MODELS

Applications of Deep Learning – Spring 2024

Rubén Martínez-Cantín, Julio Marco

Master in Robotics, Graphics and Computer Vision – Universidad de Zaragoza

Content generation

First lecture

VAEs, GANs

NEURAL
RENDERING

PHYSICALLY
BASED
RENDERING

Prior
Knowledge

Data



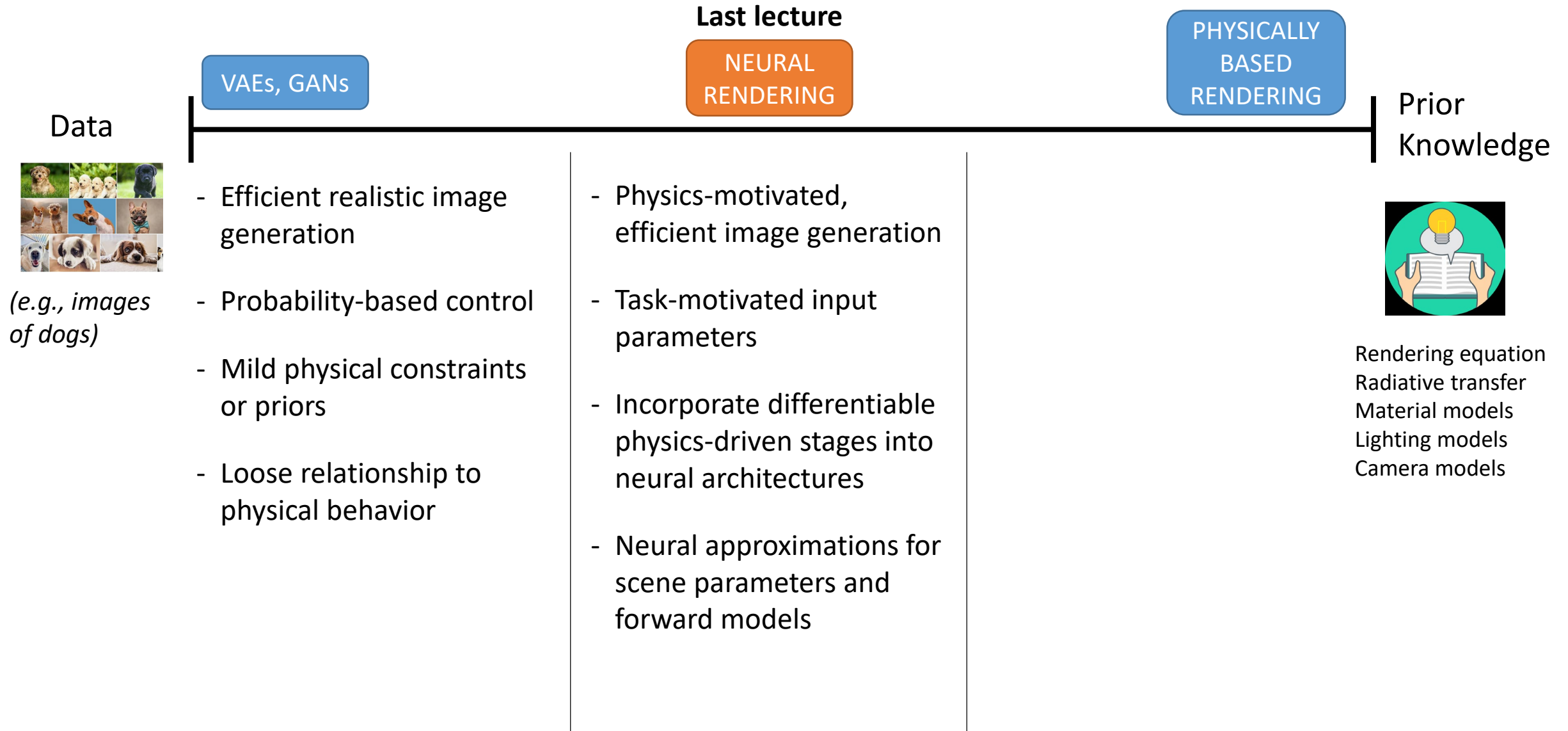
(e.g., images
of dogs)

- Efficient realistic image generation
- Probability-based control
- Mild physical constraints or priors
- Loose relationship to physical behavior



Rendering equation
Radiative transfer
Material models
Lighting models
Camera models

Content generation



Differentiable forward models: Neural rendering example

NeRF: Neural radiance fields

[Mildenhall et al. 2020]

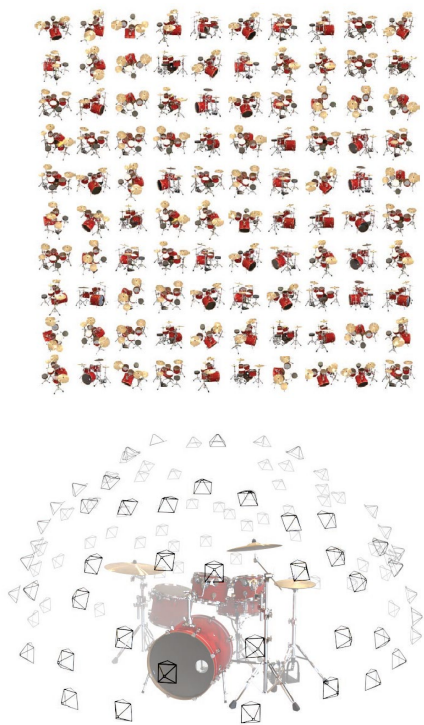
Known data



NeRF: Neural radiance fields

[Mildenhall et al. 2020]

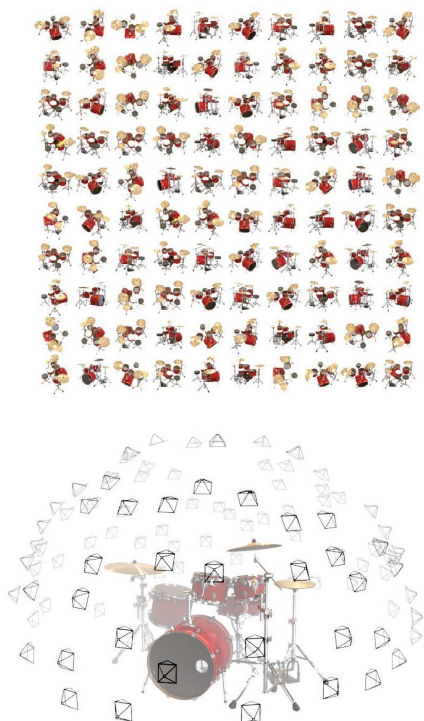
**Sparse images +
Viewpoint (5D)**



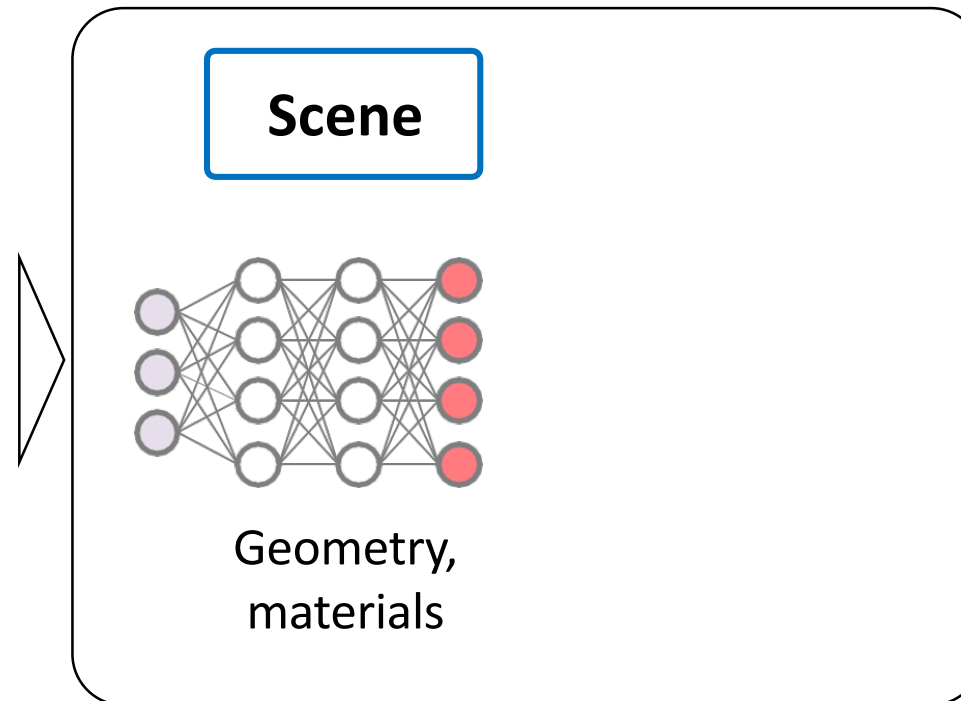
NeRF: Neural radiance fields

[Mildenhall et al. 2020]

**Sparse images +
Viewpoint (5D)**



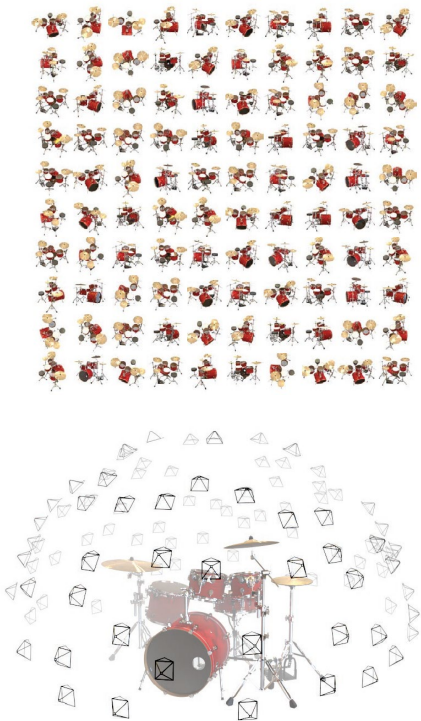
Neural architecture



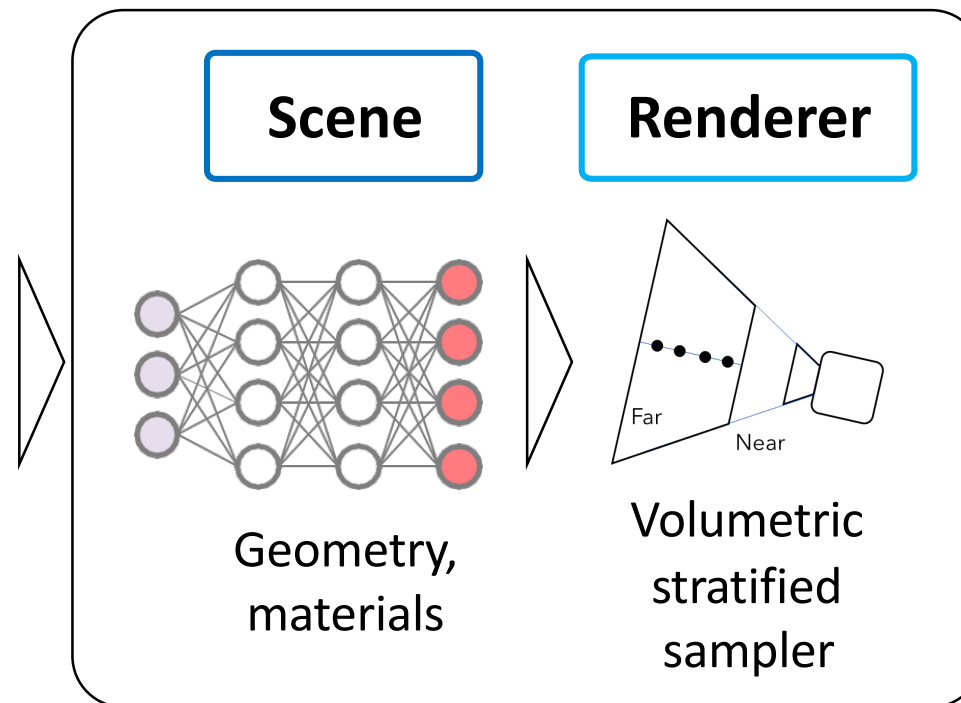
NeRF: Neural radiance fields

[Mildenhall et al. 2020]

**Sparse images +
Viewpoint (5D)**



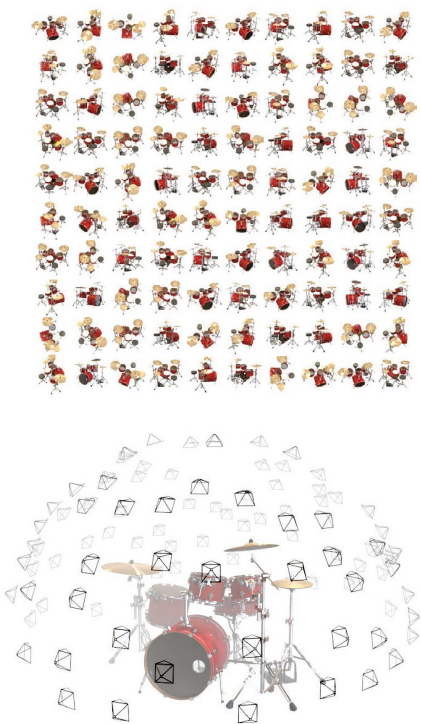
Neural architecture



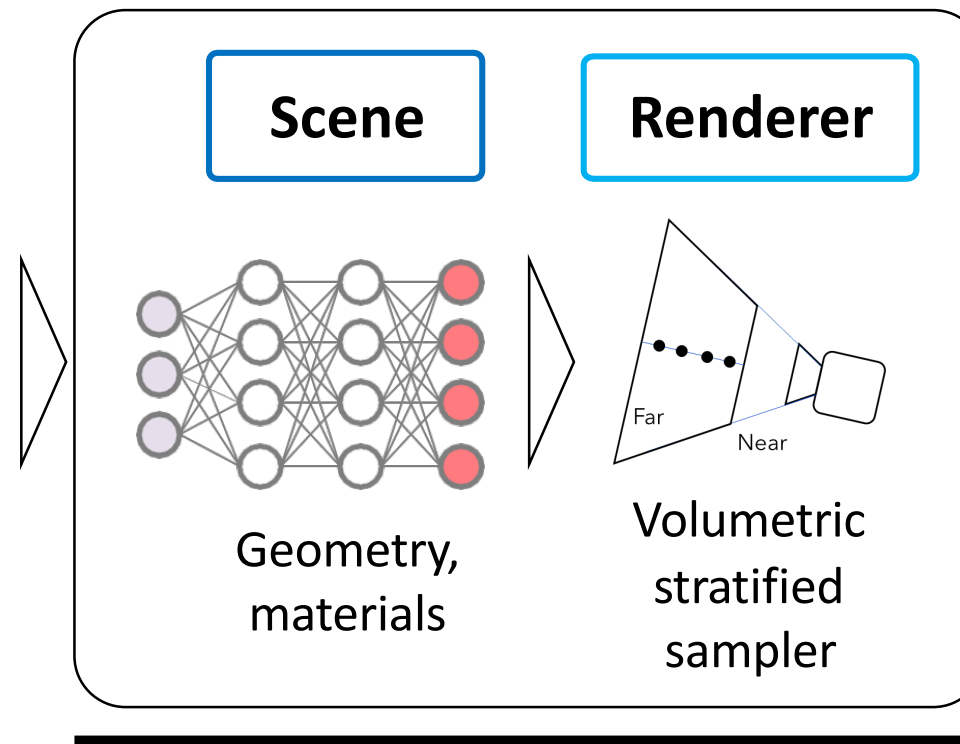
NeRF: Neural radiance fields

[Mildenhall et al. 2020]

**Sparse images +
Viewpoint (5D)**



Neural architecture

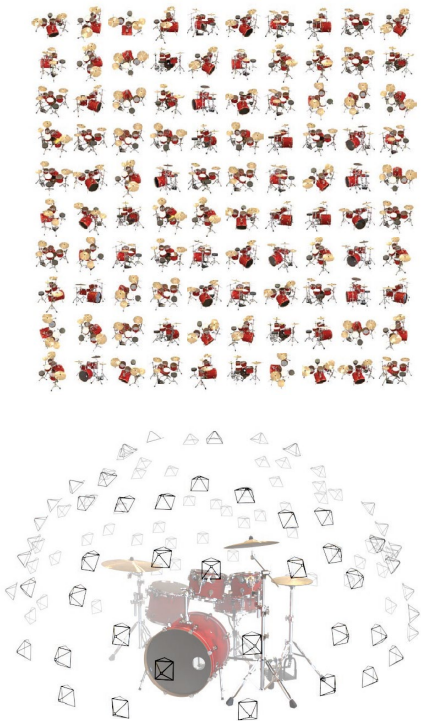


DIFFERENTIABLE

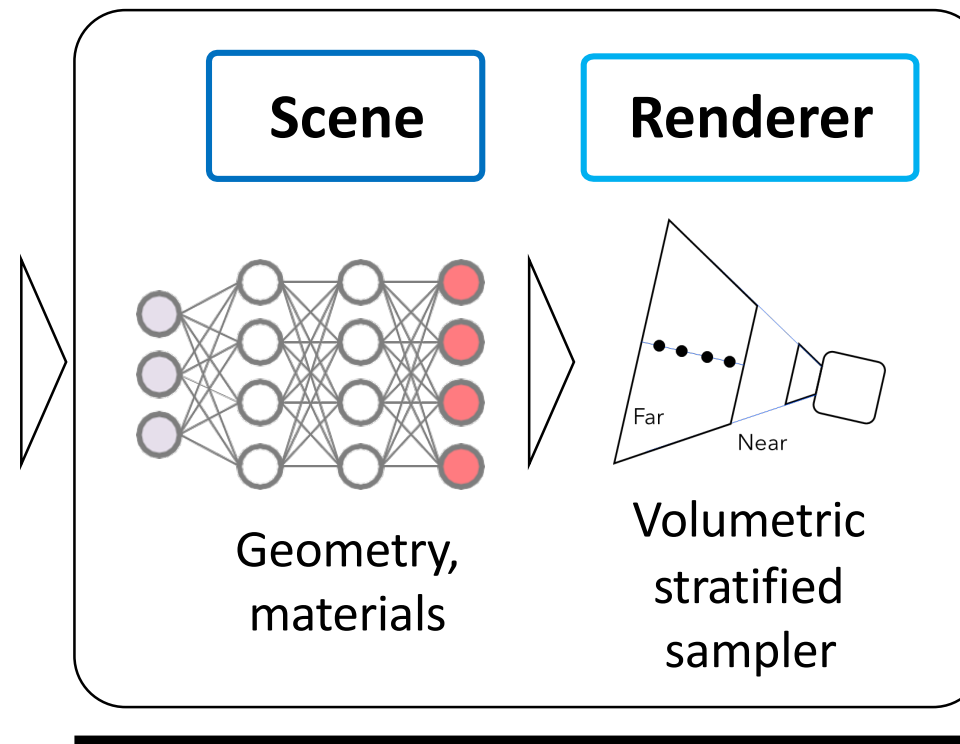
NeRF: Neural radiance fields

[Mildenhall et al. 2020]

Sparse images +
Viewpoint (5D)



Neural architecture



DIFFERENTIABLE

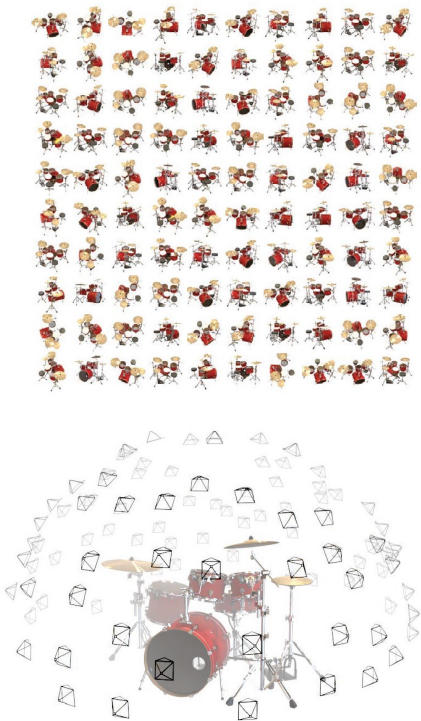
Novel views



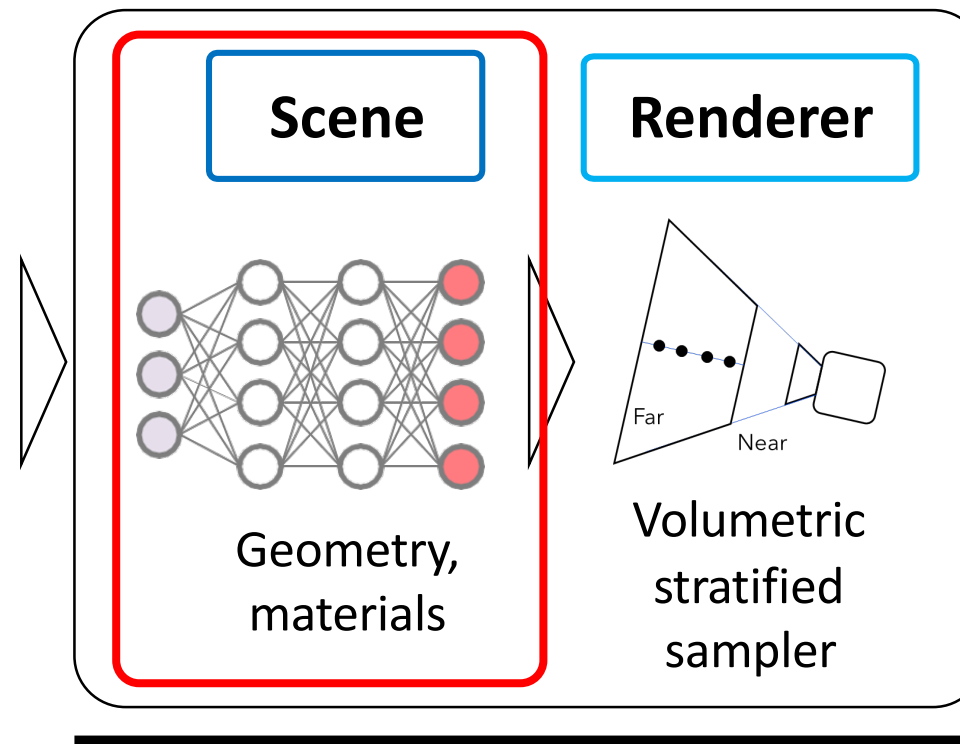
NeRF: Neural radiance fields

[Mildenhall et al. 2020]

Sparse images +
Viewpoint (5D)



Neural architecture



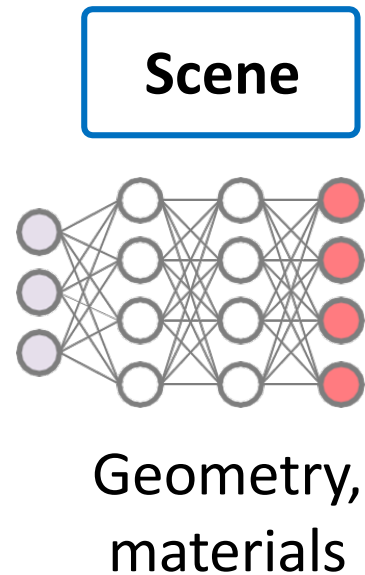
DIFFERENTIABLE

Novel views



NeRF: Neural radiance fields

Implicit scene representation

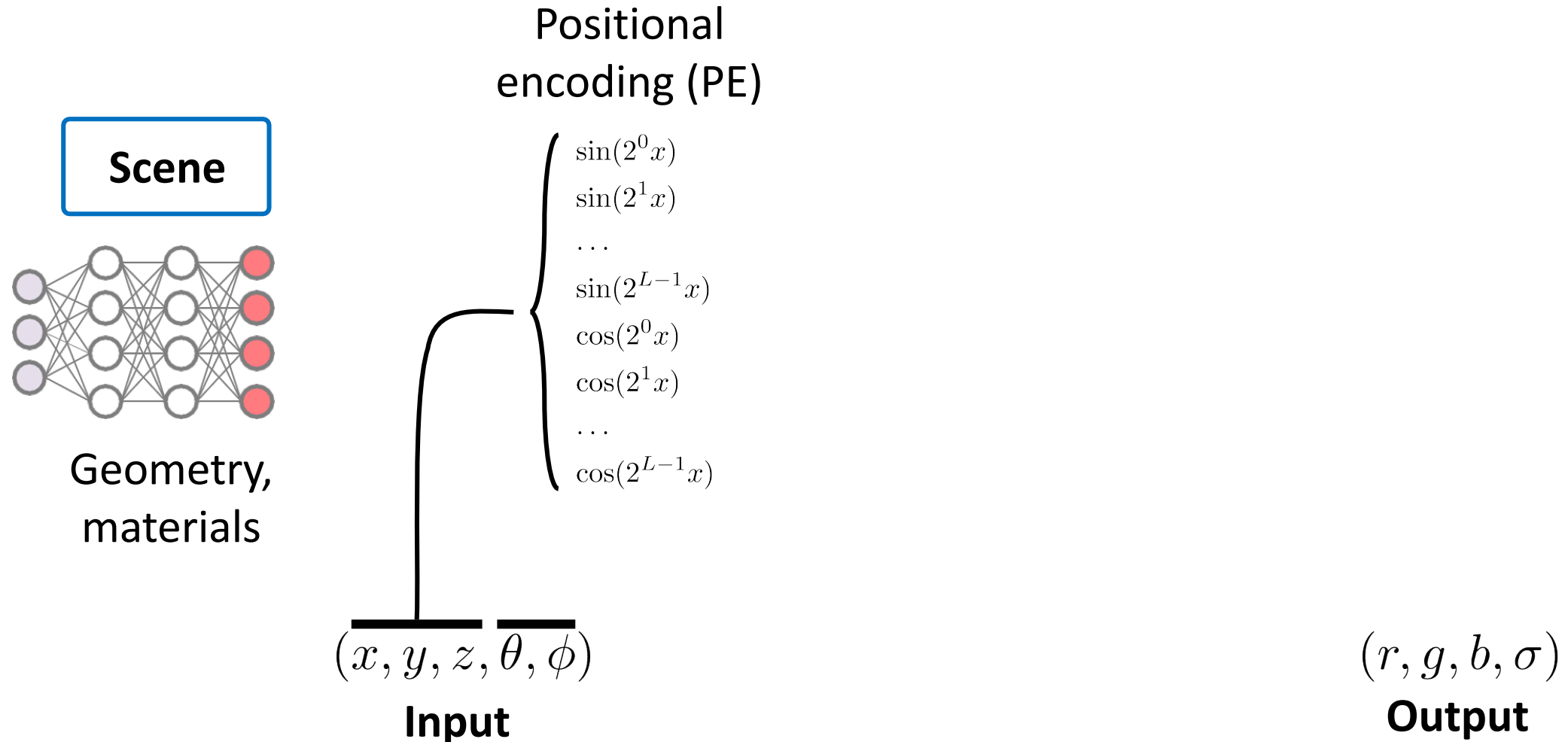


$$\begin{array}{ccc} \text{Input} & & \text{Output} \\ \hline (x, y, z, \theta, \phi) & \rightarrow & (r, g, b, \sigma) \\ \hline \text{Spatial coordinate} & \text{View direction} & \text{Color} \quad \text{Density} \end{array}$$

Represents geometry
and materials

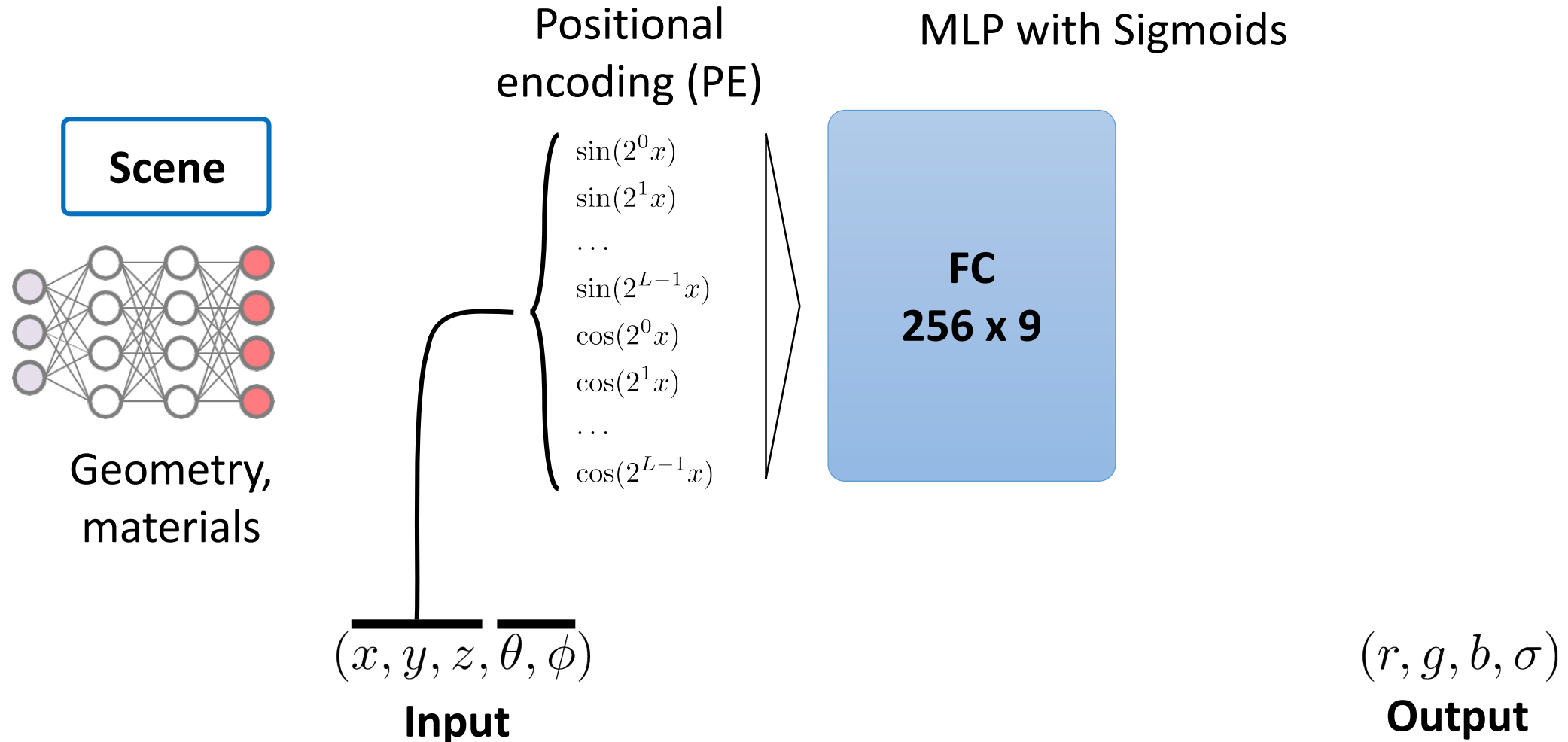
NeRF: Neural radiance fields

Implicit scene representation



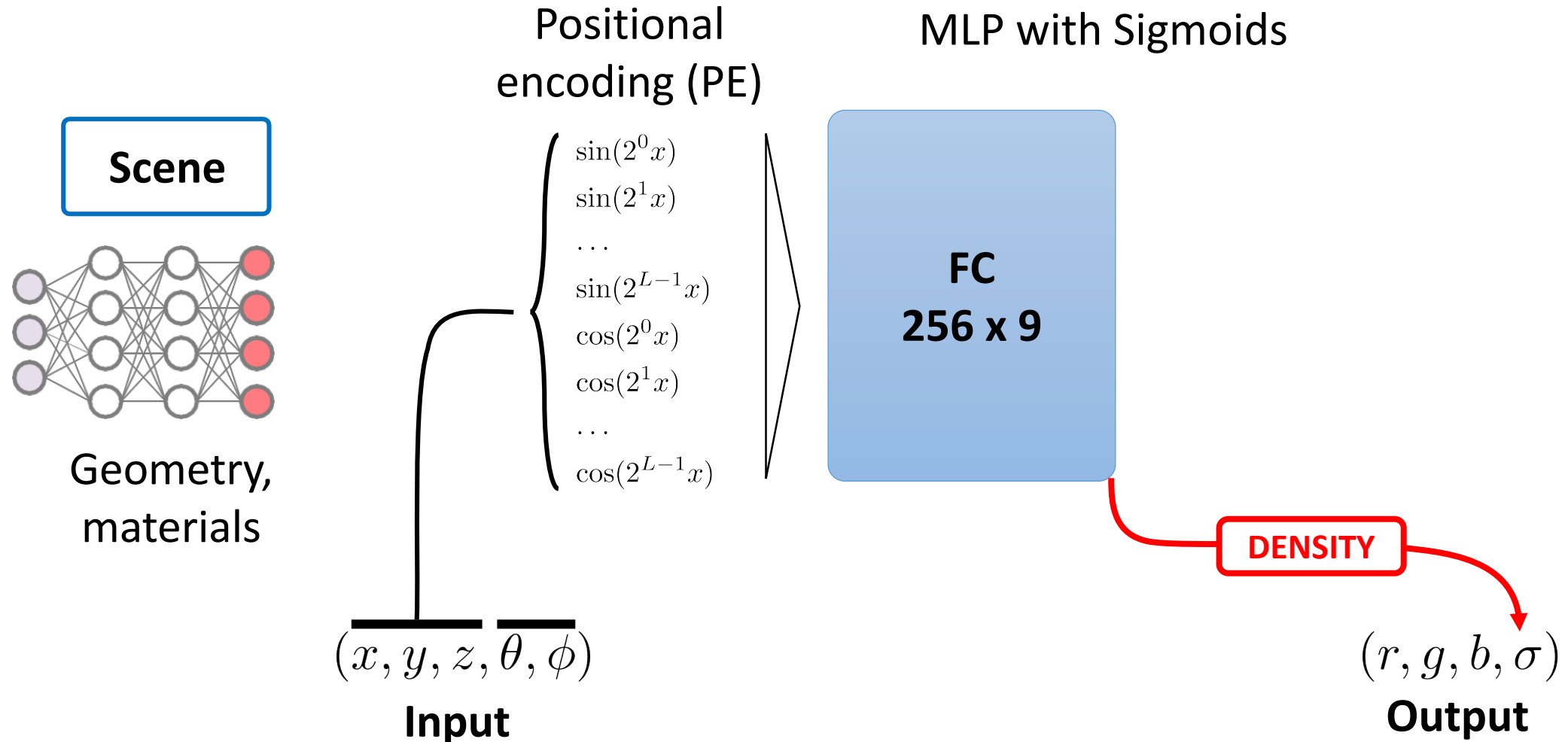
NeRF: Neural radiance fields

Implicit scene representation



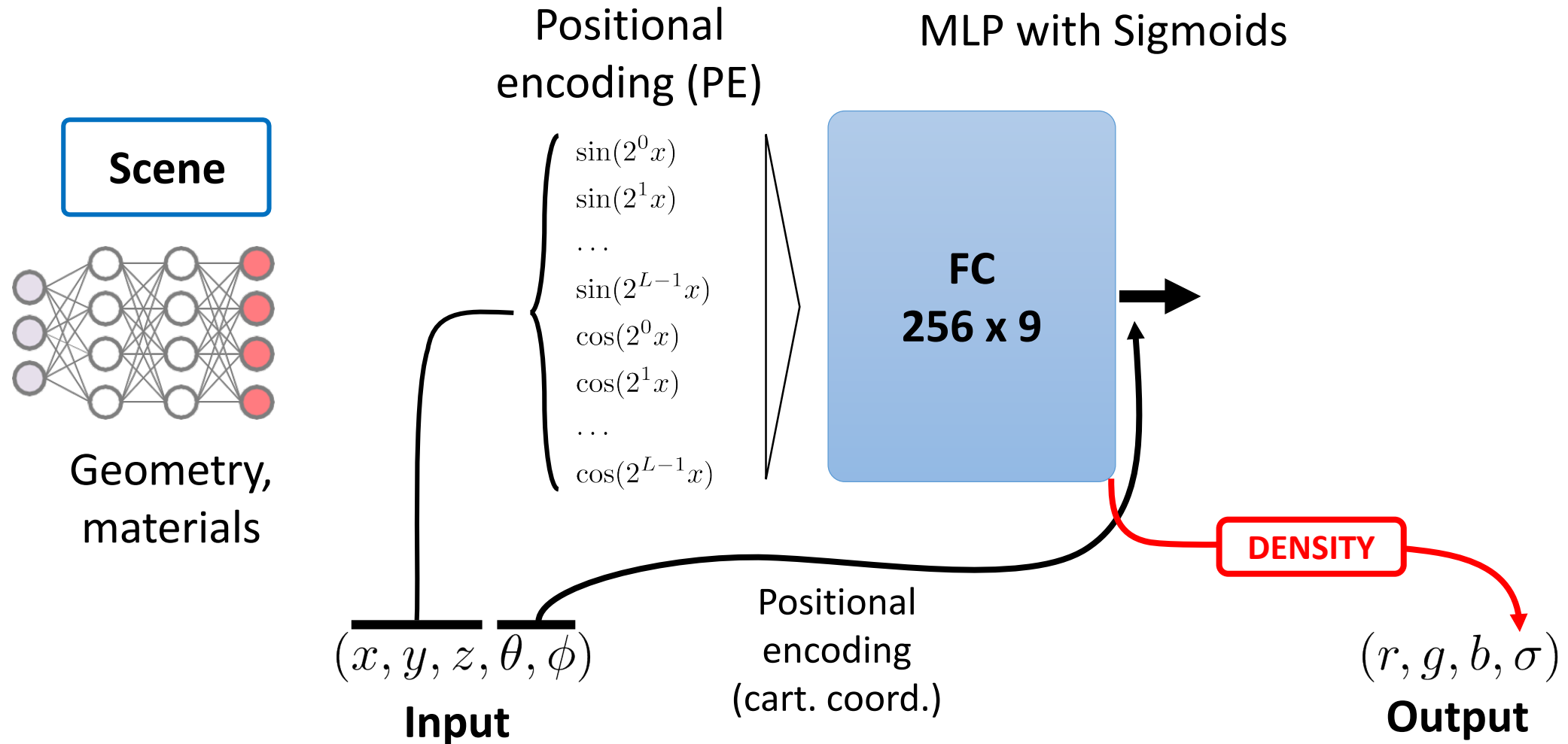
NeRF: Neural radiance fields

Implicit scene representation



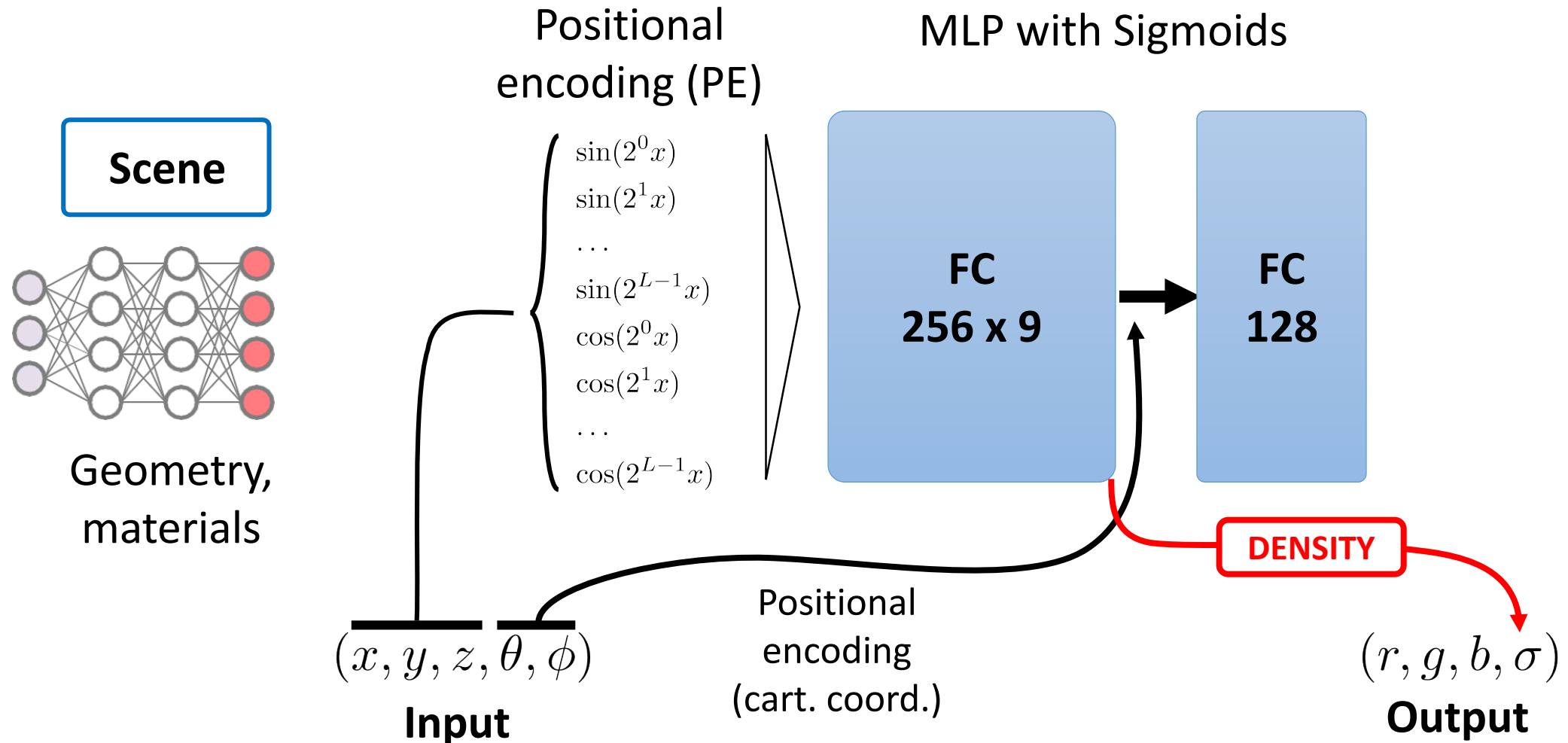
NeRF: Neural radiance fields

Implicit scene representation



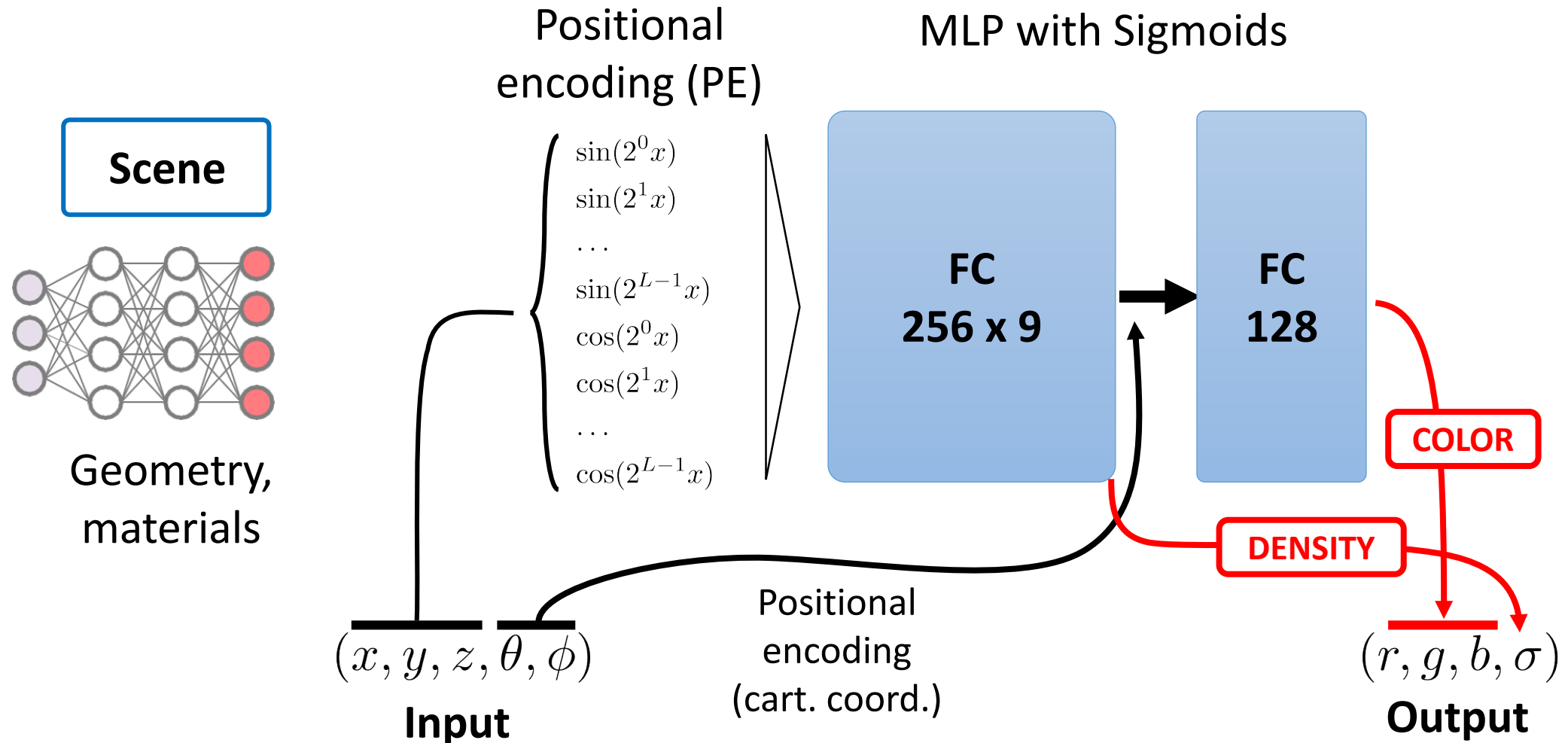
NeRF: Neural radiance fields

Implicit scene representation



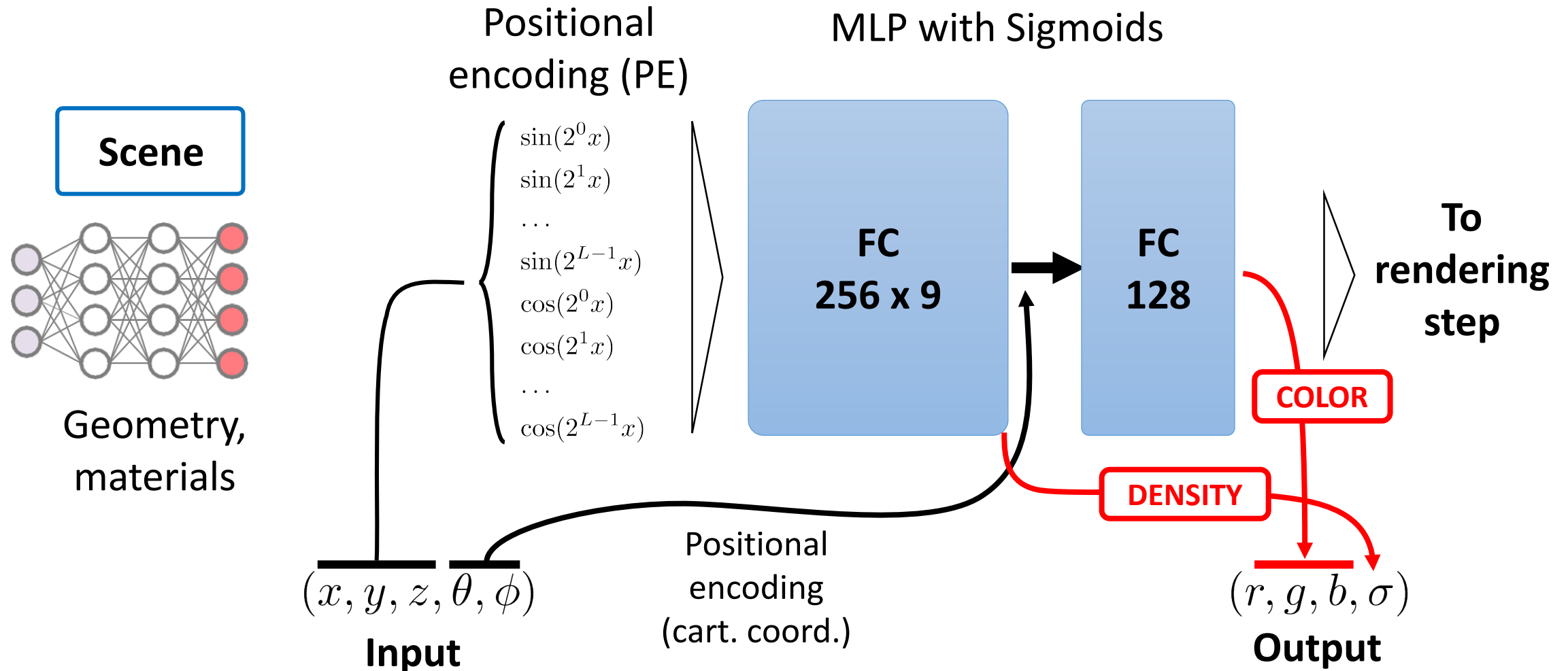
NeRF: Neural radiance fields

Implicit scene representation



NeRF: Neural radiance fields

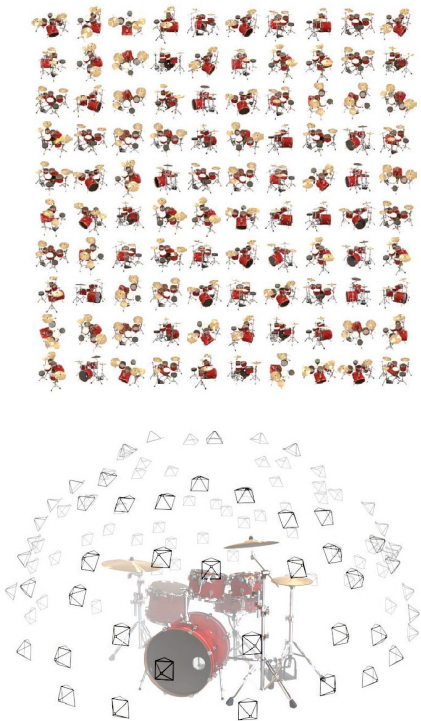
Implicit scene representation



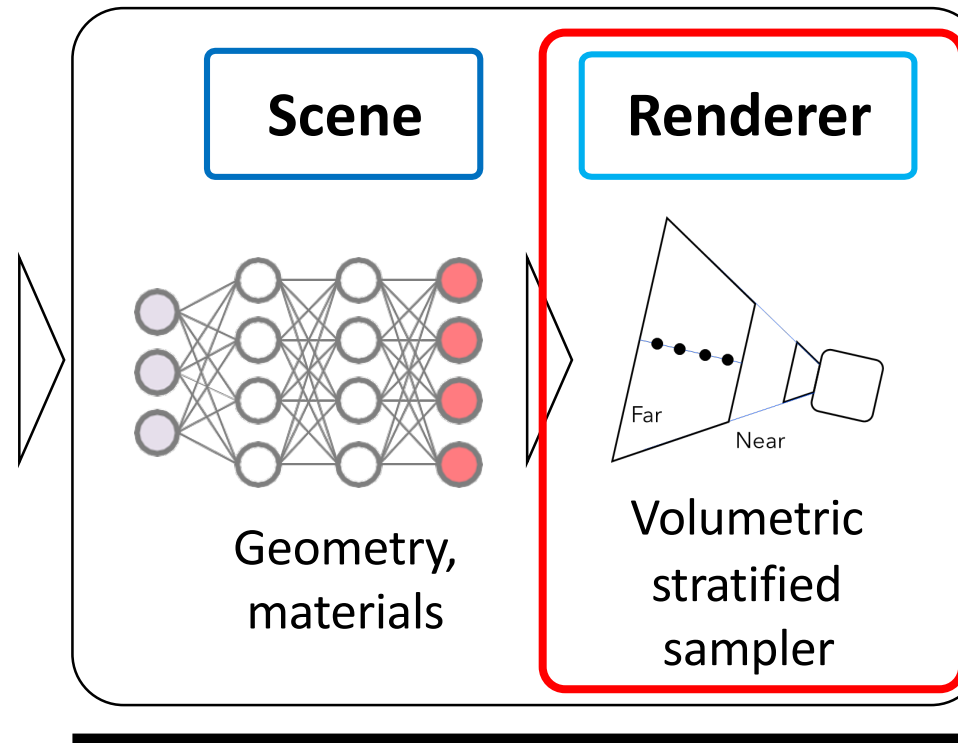
NeRF: Neural radiance fields

[Mildenhall et al. 2020]

Sparse images +
Viewpoint (5D)



Neural architecture



DIFFERENTIABLE

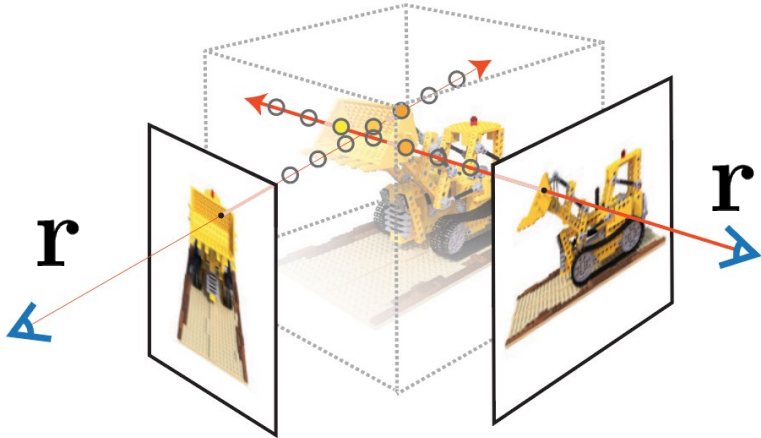
Novel views



NeRF: Neural radiance fields

Volumetric renderer

Renderer

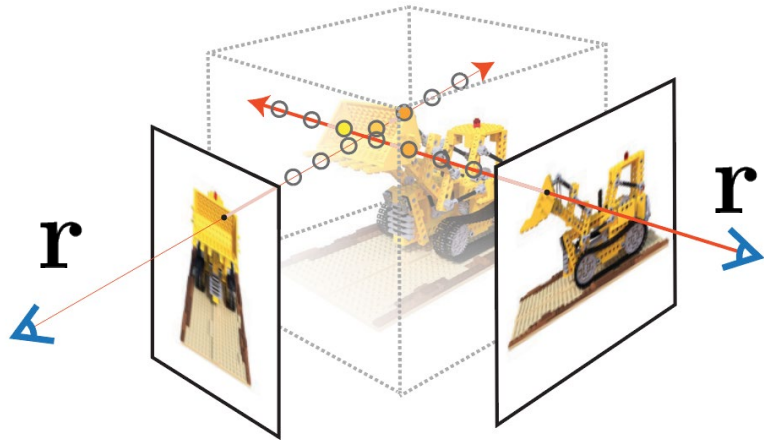


Volumetric
stratified sampler

NeRF: Neural radiance fields

Volumetric renderer

Renderer



Volumetric
stratified sampler

Output color

$$\overline{C(\mathbf{r})} =$$



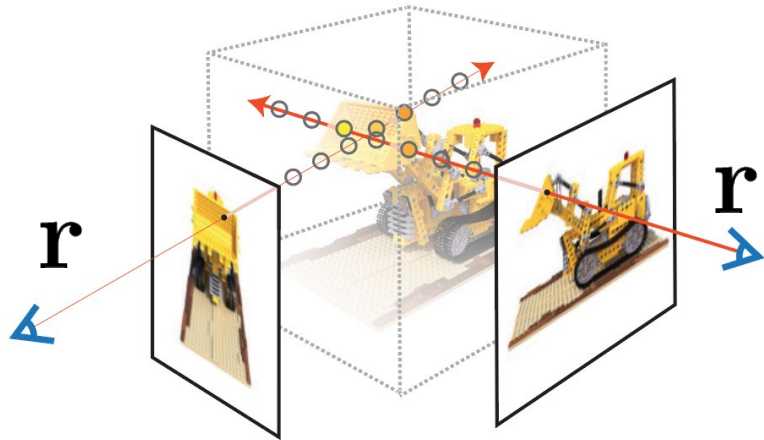
Ray with
direction

$\mathbf{d}$

NeRF: Neural radiance fields

Volumetric renderer

Renderer



Volumetric
stratified sampler

Forward model, ray integral

Output color

$$\overline{C(\mathbf{r})} = \int_{t_n}^{t_f} T(t) \underbrace{\sigma(\mathbf{r}(t))}_{\text{Density}} \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt$$

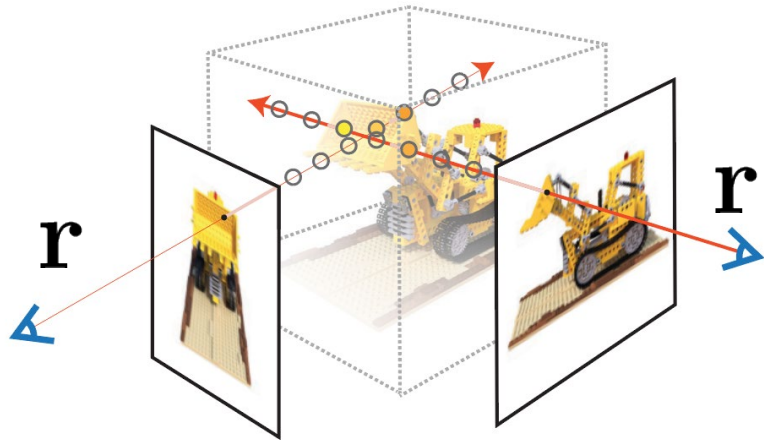
Ray with
direction
 $\mathbf{d}$

$$\mathbf{r}(t) \equiv (x, y, z)$$

NeRF: Neural radiance fields

Volumetric renderer

Renderer



Volumetric
stratified sampler

Forward model, ray integral

Output color

$$\overline{C(\mathbf{r})} = \int_{t_n}^{t_f} T(t) \underbrace{\sigma(\mathbf{r}(t))}_{\text{Density}} \underbrace{\mathbf{c}(\mathbf{r}(t), \mathbf{d})}_{\text{Color}} dt$$

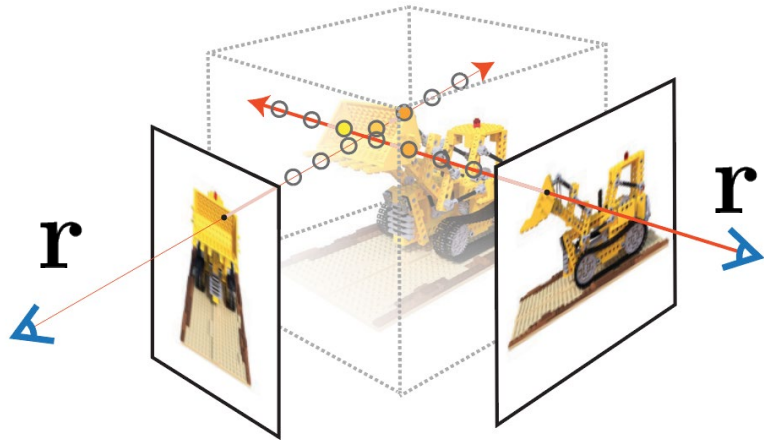
Ray with
direction
 $\mathbf{d}$

$$\mathbf{r}(t) \equiv (x, y, z)$$

NeRF: Neural radiance fields

Volumetric renderer

Renderer



Volumetric
stratified sampler

Forward model, ray integral

Output color

$$\overline{C(\mathbf{r})} = \int_{t_n}^{t_f} \overline{T(t)} \underbrace{\sigma(\mathbf{r}(t))}_{\text{Density}} \underbrace{\mathbf{c}(\mathbf{r}(t), \mathbf{d})}_{\text{Color}} dt$$

↑

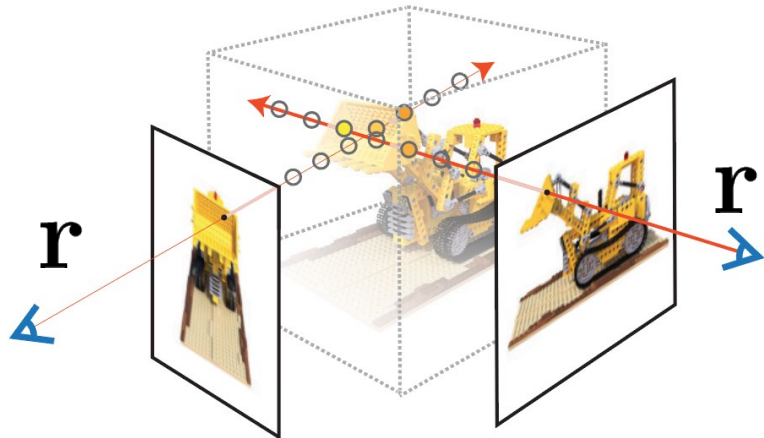
Ray with
direction
 $\mathbf{d}$

$$\mathbf{r}(t) \equiv (x, y, z)$$

NeRF: Neural radiance fields

Volumetric renderer

Renderer



Volumetric
stratified sampler

Forward model, **quadrature estimation**

Output color

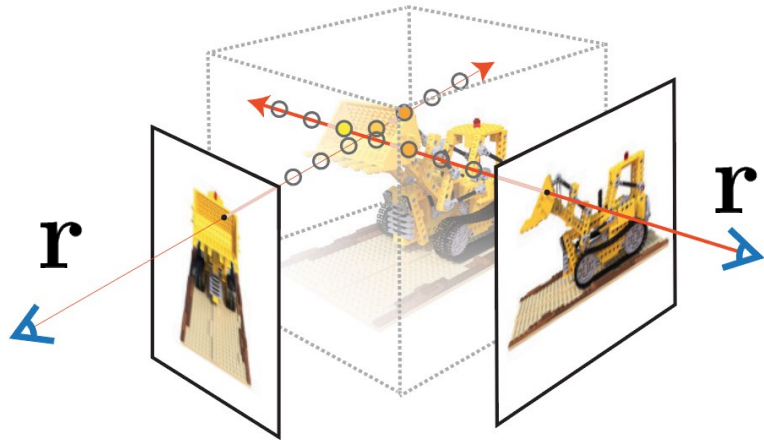
$$\hat{\mathbf{C}}(\mathbf{r}) = \sum_{i=1}^N \overbrace{T_i}^{\text{Transmittance}} (1 - \exp(-\underbrace{\sigma_i}_{\text{Density}} \delta_i)) \underbrace{\mathbf{c}_i}_{\text{Color}}$$

Ray with direction $\mathbf{d}$

NeRF: Neural radiance fields

Volumetric renderer

Renderer



Volumetric
stratified sampler

Forward model, quadrature estimation

Output color

$$\hat{\underline{C}}(\mathbf{r}) = \sum_{i=1}^N \underbrace{T_i}_{\text{Density}} (1 - \exp(-\underbrace{\sigma_i \delta_i}_{\text{Color}}))$$

Ray with
direction
 $\mathbf{d}$

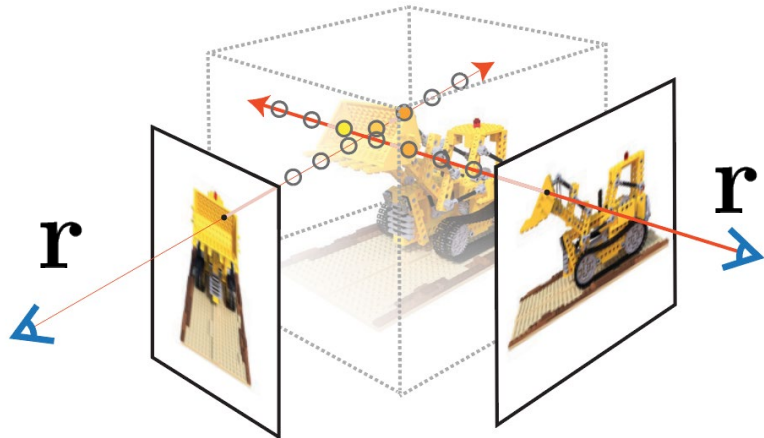
Transmittance $T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right)$

NeRF: Neural radiance fields

Volumetric renderer

Closed form & Differentiable

Renderer



Volumetric
stratified sampler

Forward model, **quadrature estimation**

Output color

$$\hat{\underline{C}}(\mathbf{r}) = \sum_{i=1}^N \underbrace{T_i}_{\text{Transmittance}} (1 - \exp(-\underbrace{\sigma_i}_{\text{Density}} \delta_i)) \underbrace{\mathbf{c}_i}_{\text{Color}}$$

Ray with direction $\mathbf{d}$

$$T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right)$$

NeRF: Neural radiance fields

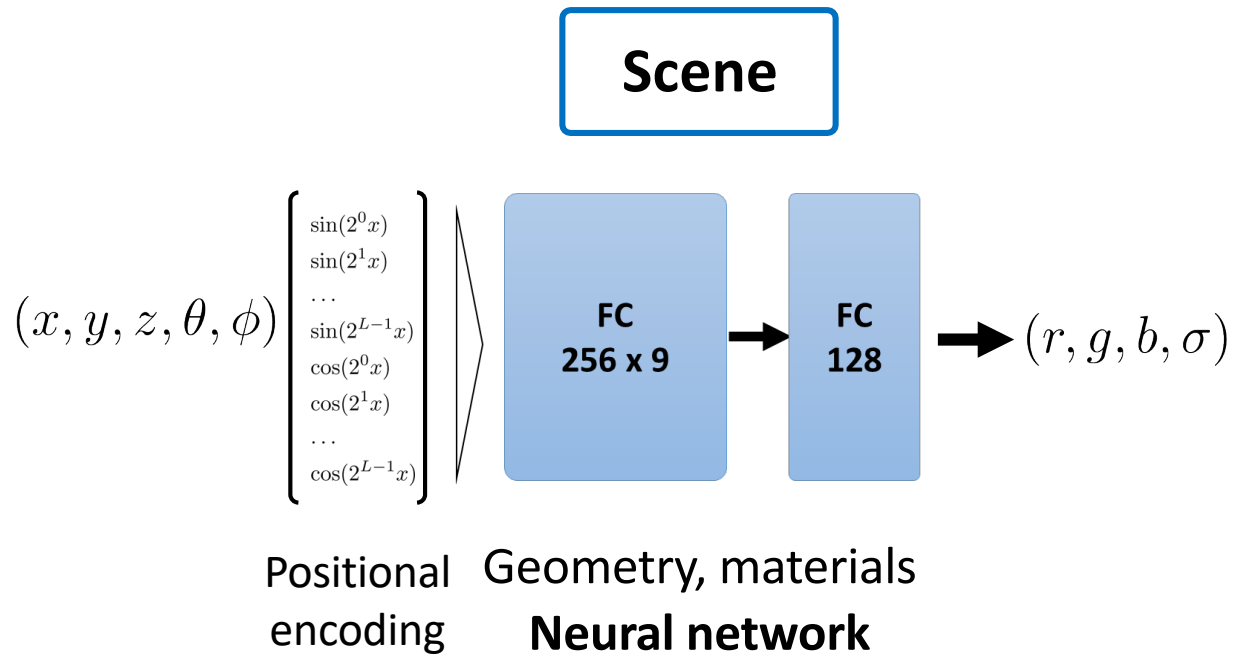
Full pipeline

$$(x, y, z, \theta, \phi) \begin{bmatrix} \sin(2^0 x) \\ \sin(2^1 x) \\ \dots \\ \sin(2^{L-1} x) \\ \cos(2^0 x) \\ \cos(2^1 x) \\ \dots \\ \cos(2^{L-1} x) \end{bmatrix}$$

Positional
encoding

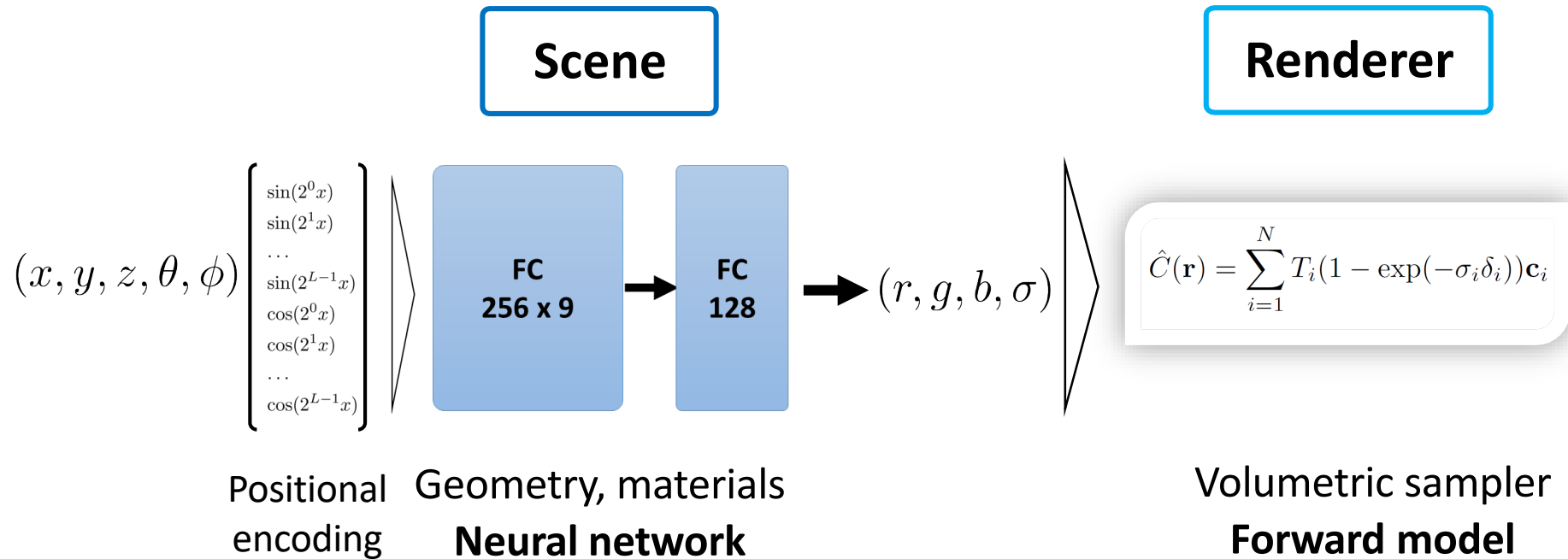
NeRF: Neural radiance fields

Full pipeline



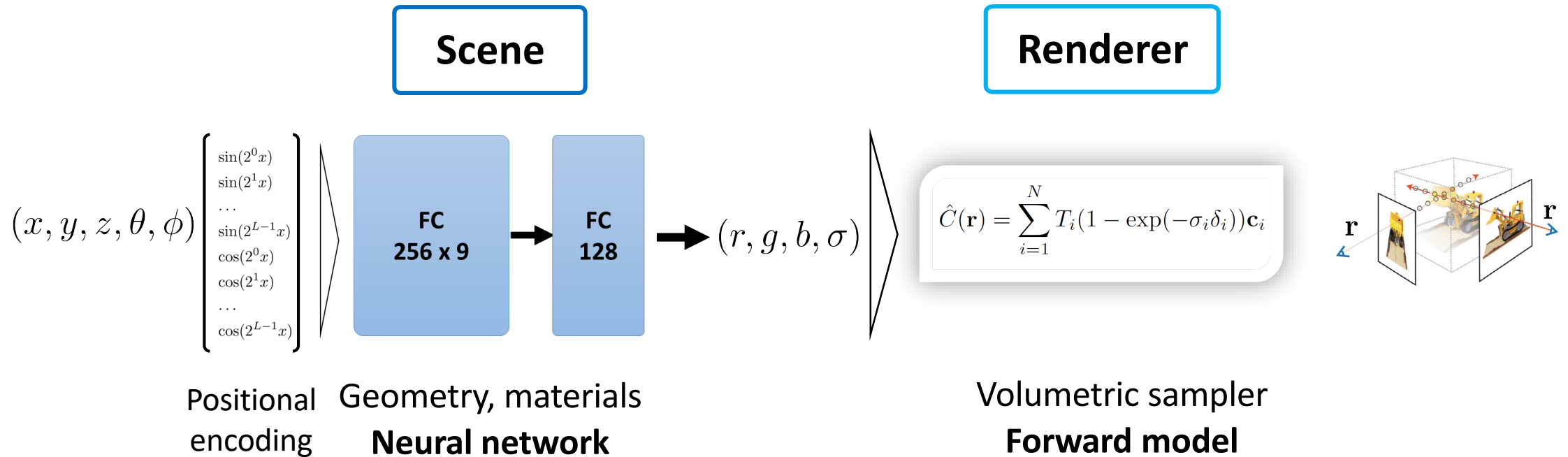
NeRF: Neural radiance fields

Full pipeline



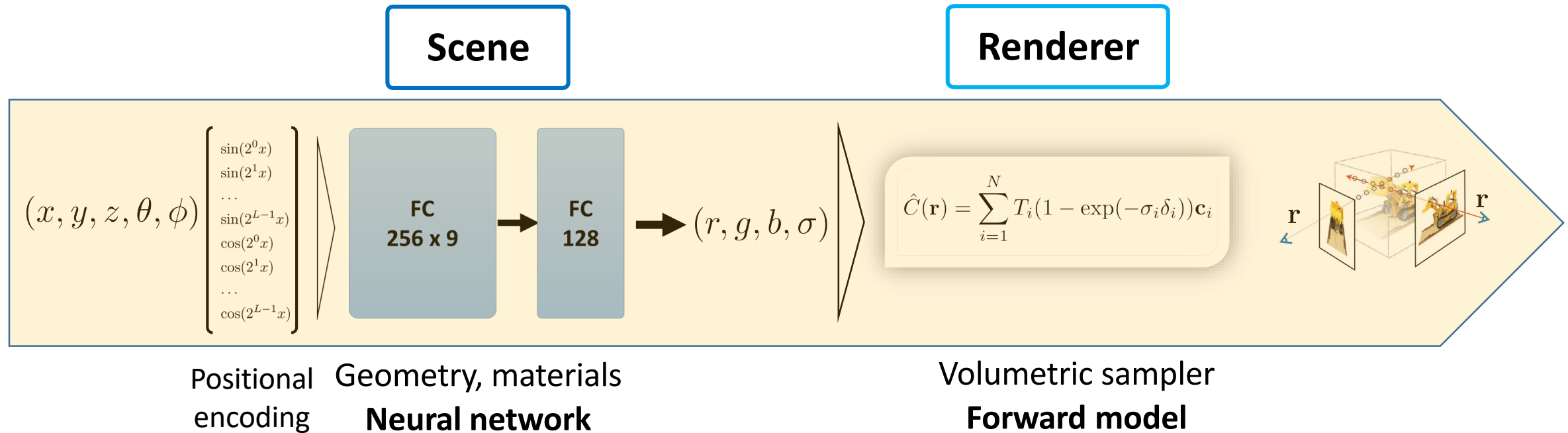
NeRF: Neural radiance fields

Full pipeline



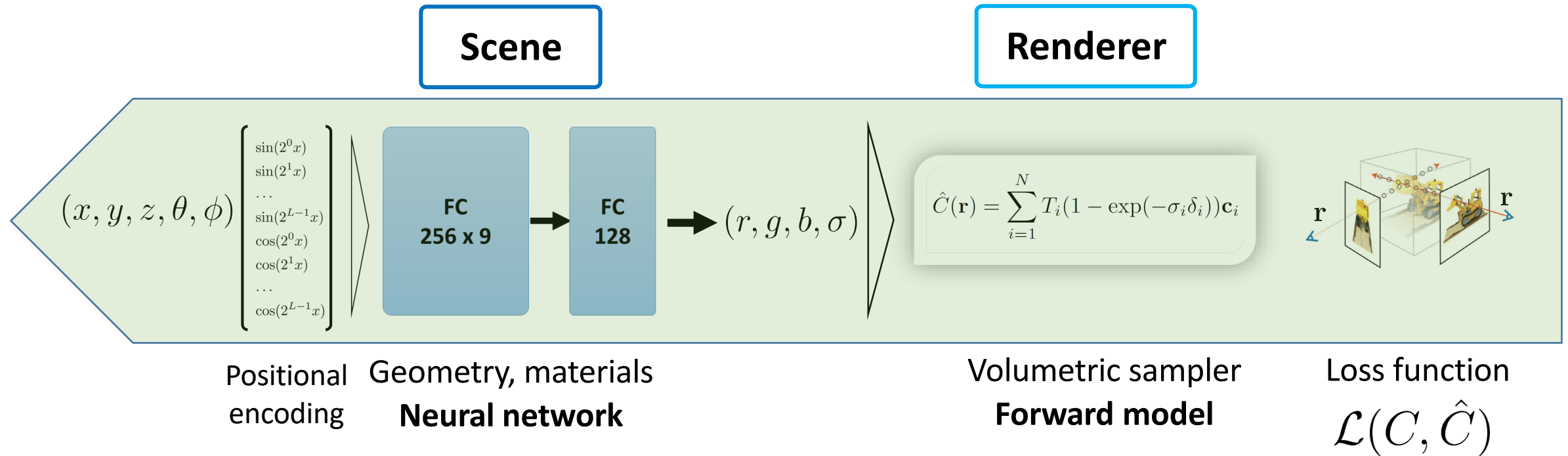
NeRF: Neural radiance fields

Full pipeline



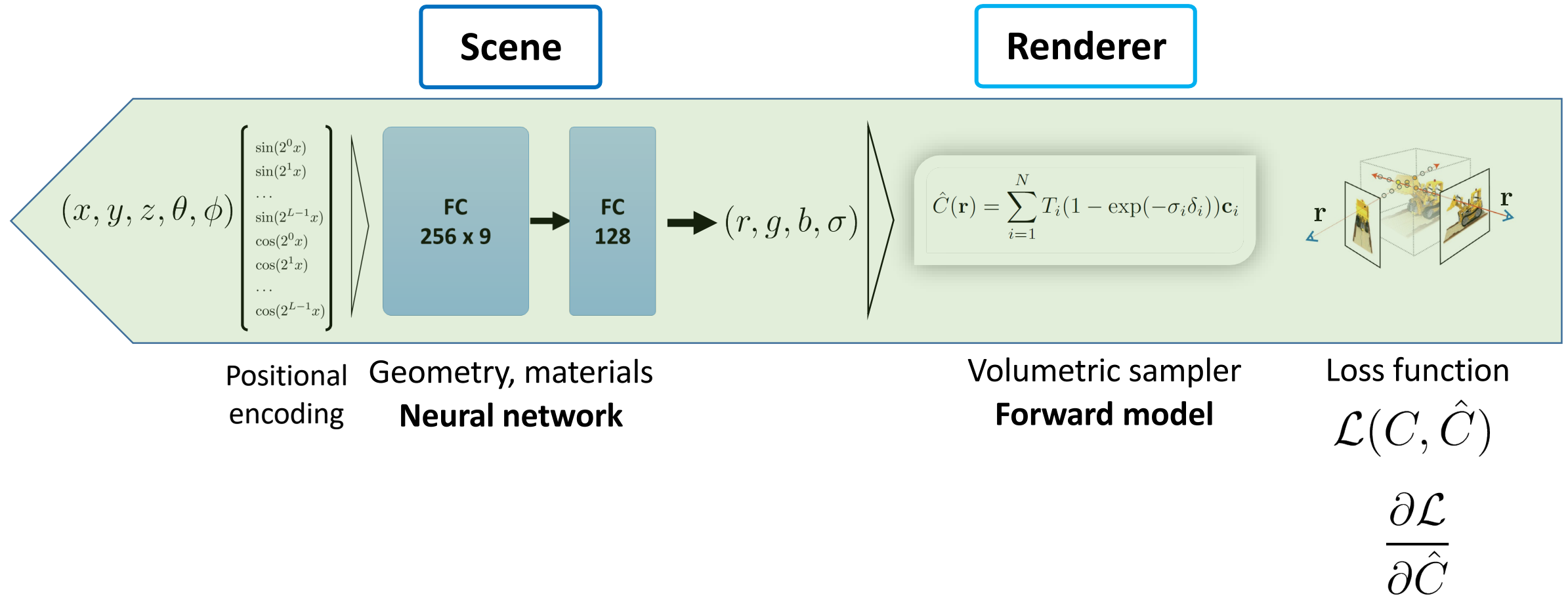
NeRF: Neural radiance fields

Full pipeline



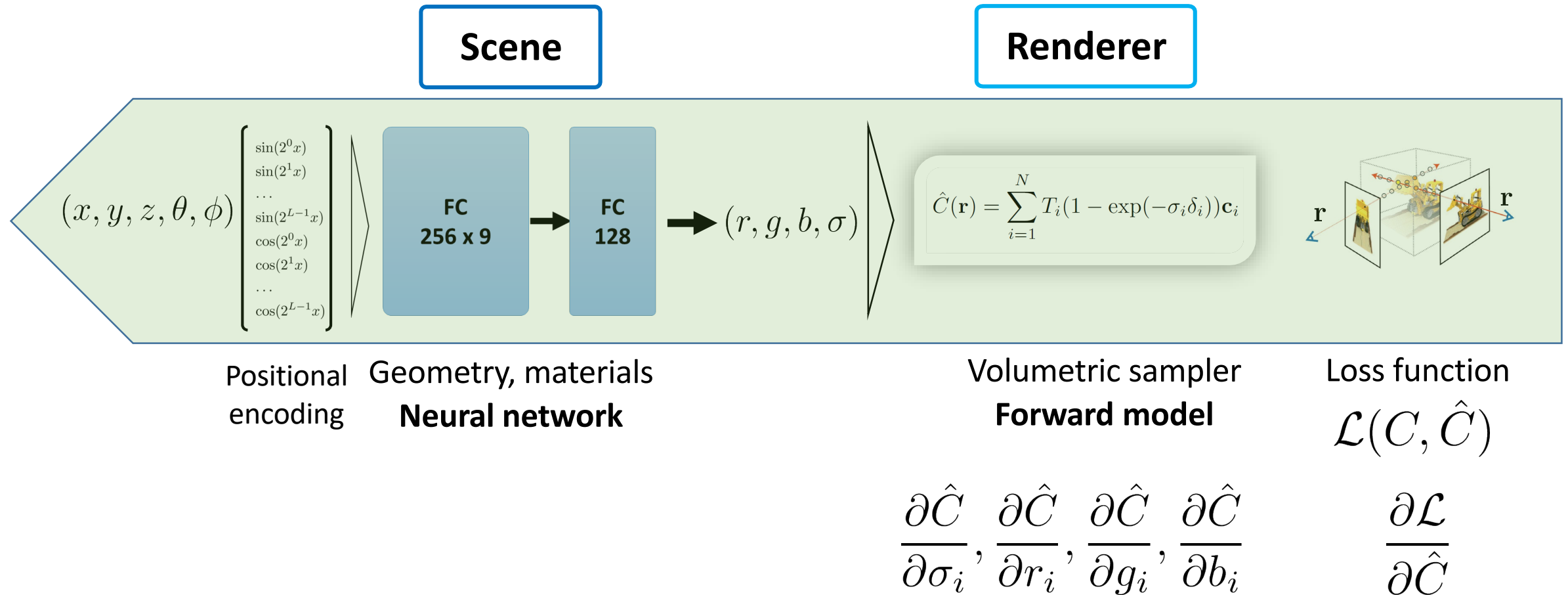
NeRF: Neural radiance fields

Full pipeline



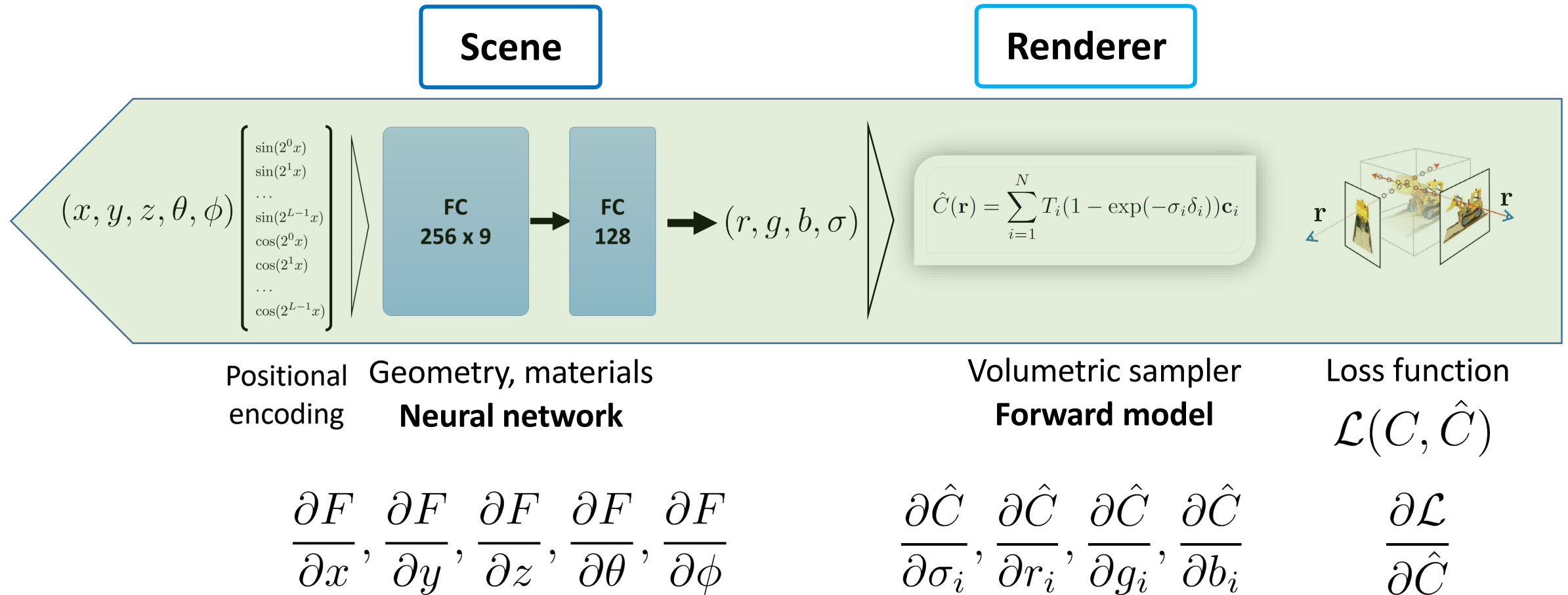
NeRF: Neural radiance fields

Full pipeline



NeRF: Neural radiance fields

Full pipeline

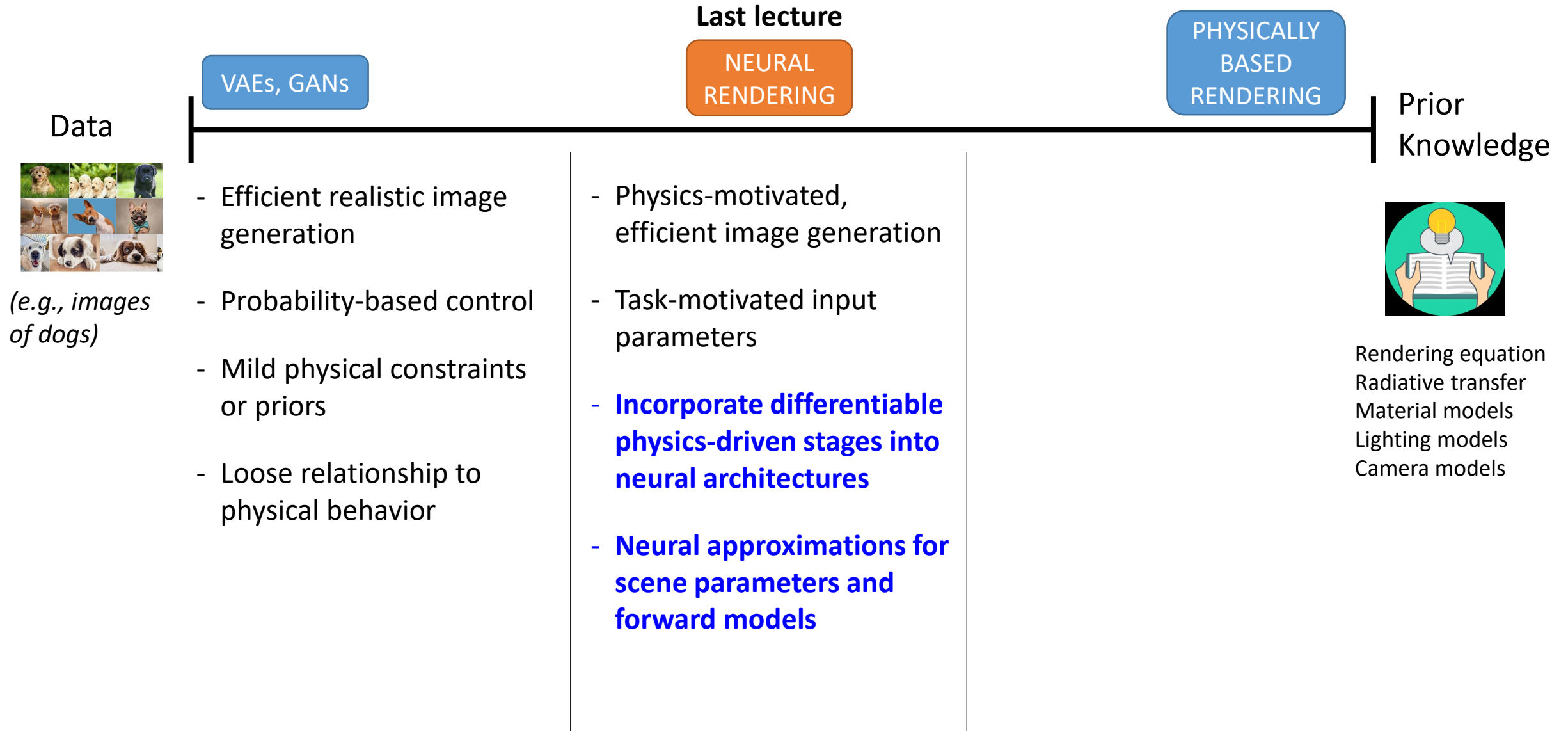


NeRF: Neural radiance fields

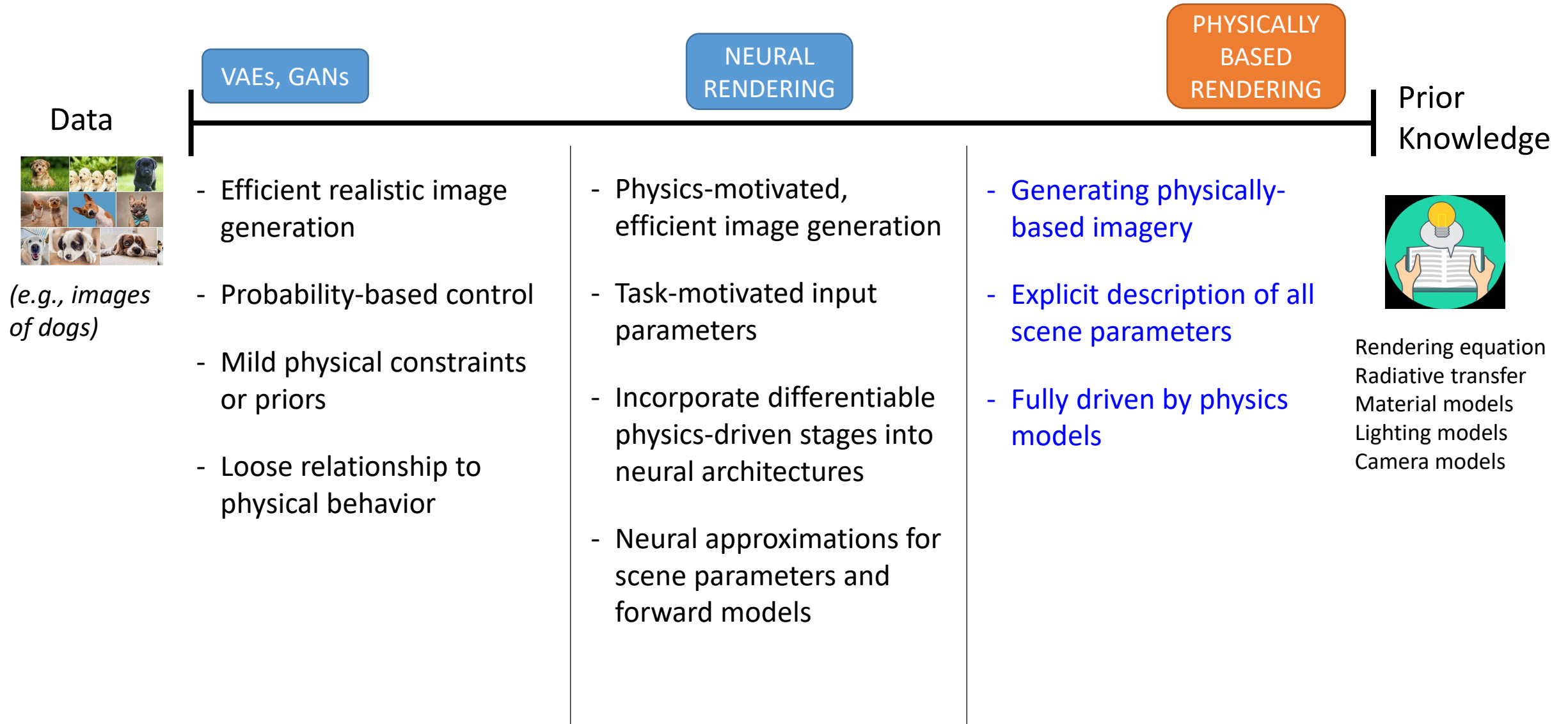
[Source](#)



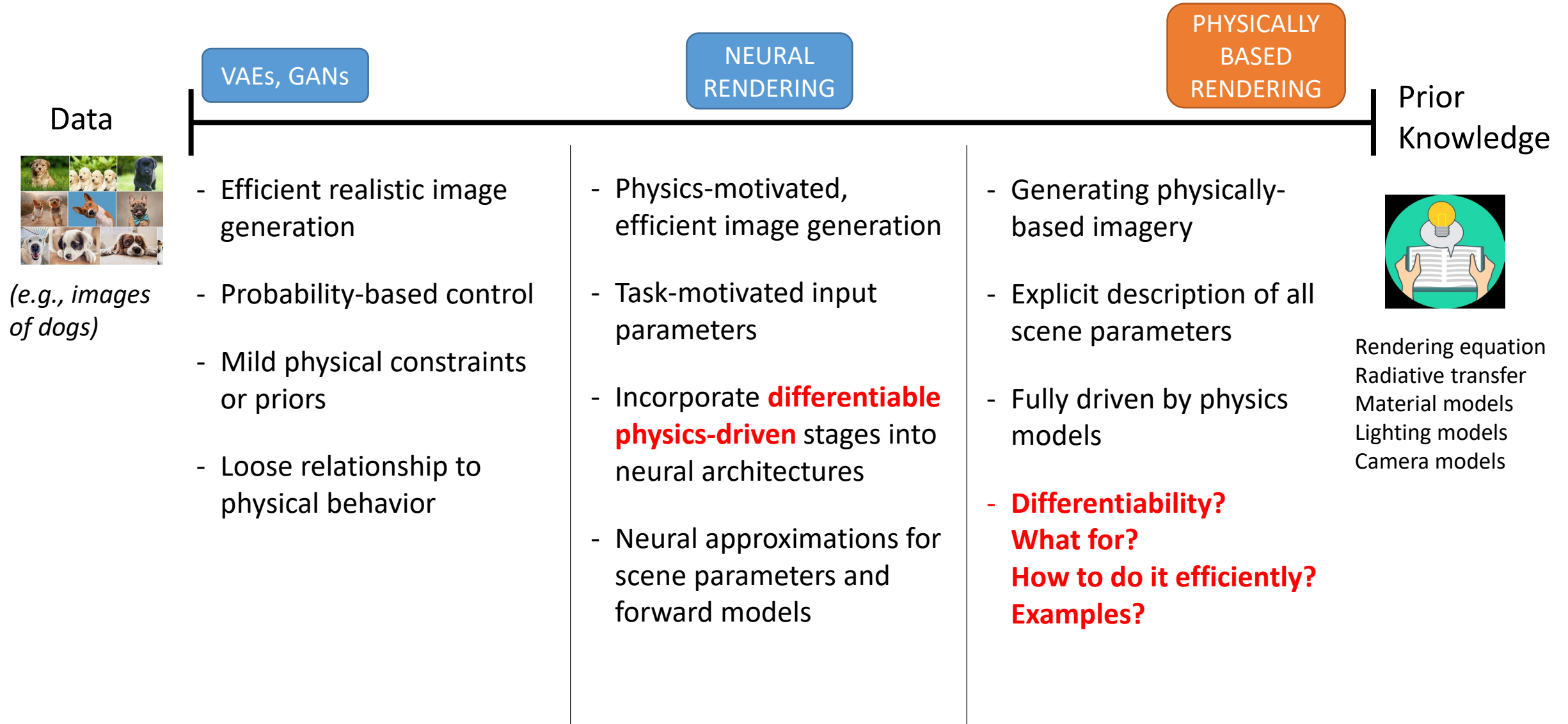
Content generation



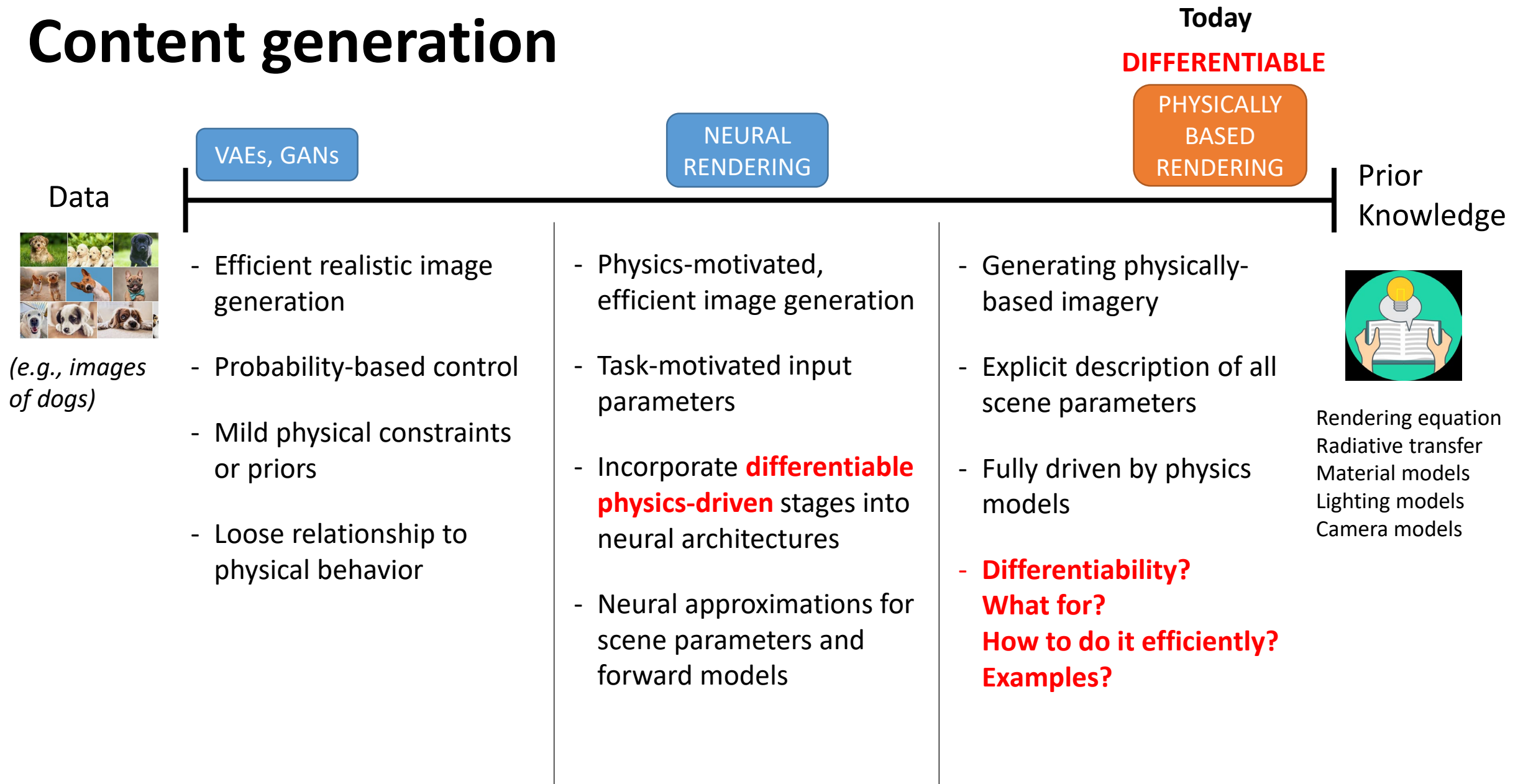
Content generation



Content generation



Content generation



Differentiable rendering

Slide credits

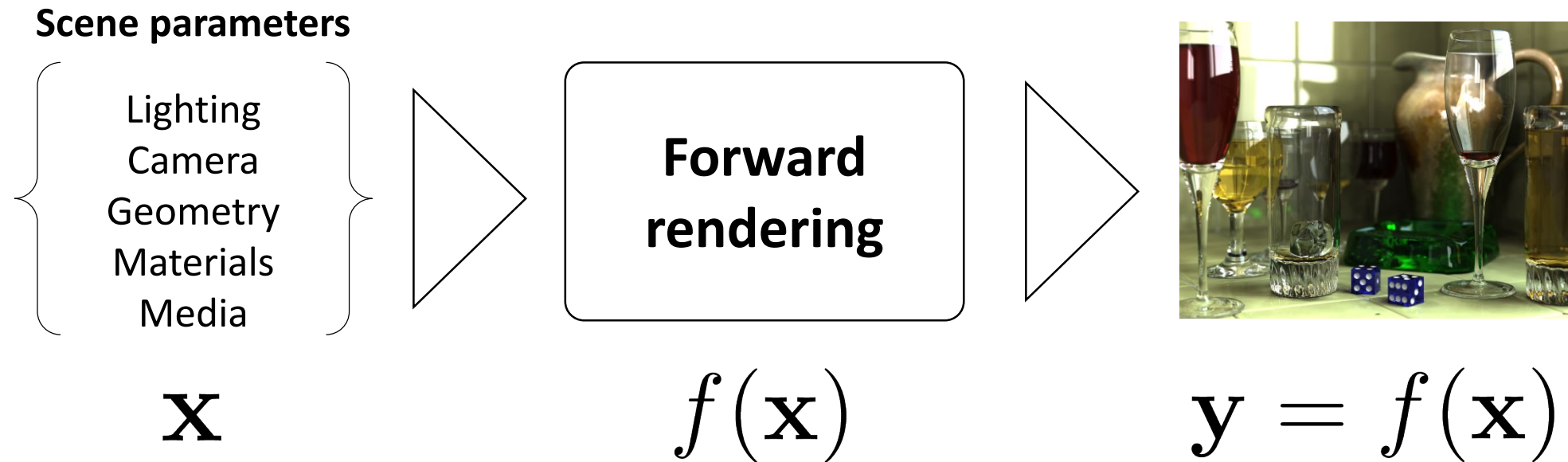
Part of these slides are based on the following presentations:

- [Mitsuba 2: A retargetable Forward and Inverse Renderer](#), [Nimier-David et al. 2019]
- [Radiative Backpropagation](#) [Nimier-David et al. 2020]
- [Wenzel Jakob's talk](#) at the *Research Seminars* course, MRGCV, Universidad de Zaragoza.

Please refer to the previous links for further information about today's topic.

Why differentiable rendering?

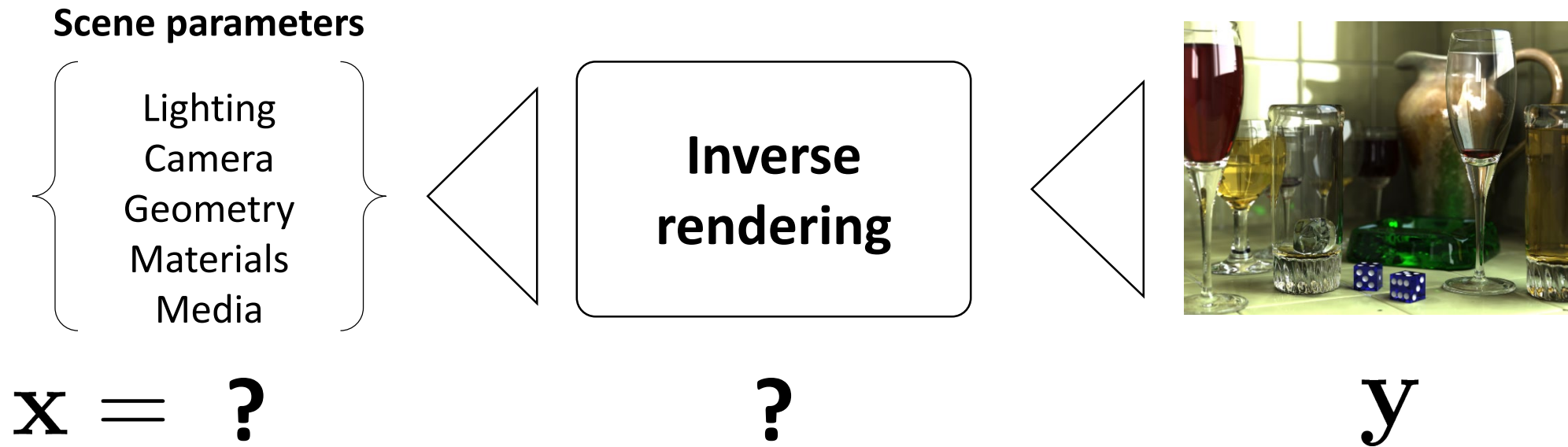
- **Forward rendering** (or just rendering):
Synthesizing an image from scene parameters



Why differentiable rendering?

- **Inverse rendering:**

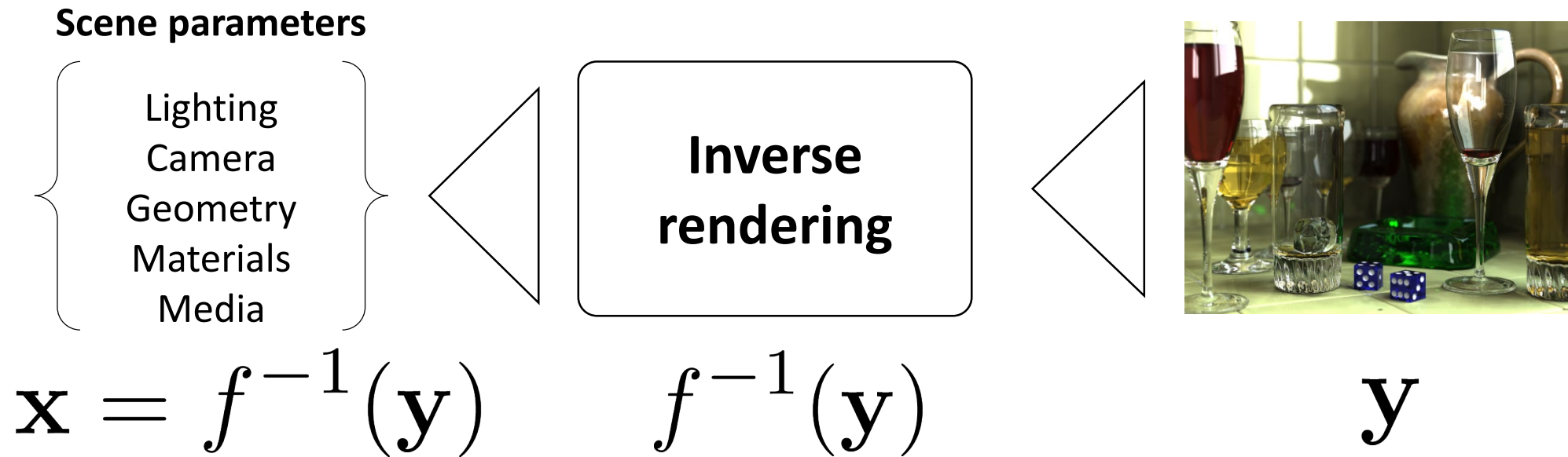
Given an image, compute the corresponding scene parameters



Why differentiable rendering?

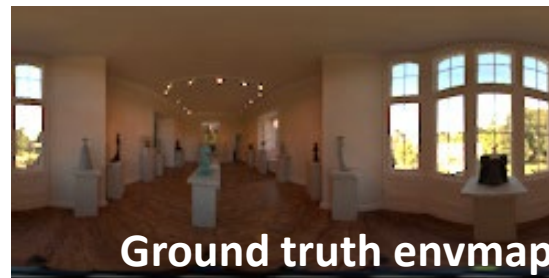
- **Inverse rendering:**

Given an image, compute the corresponding scene parameters



Why differentiable rendering?

Illumination reconstruction



Example from [Mitsuba 2](#)



Why differentiable rendering?

Shape and material
reconstruction



Initial scene

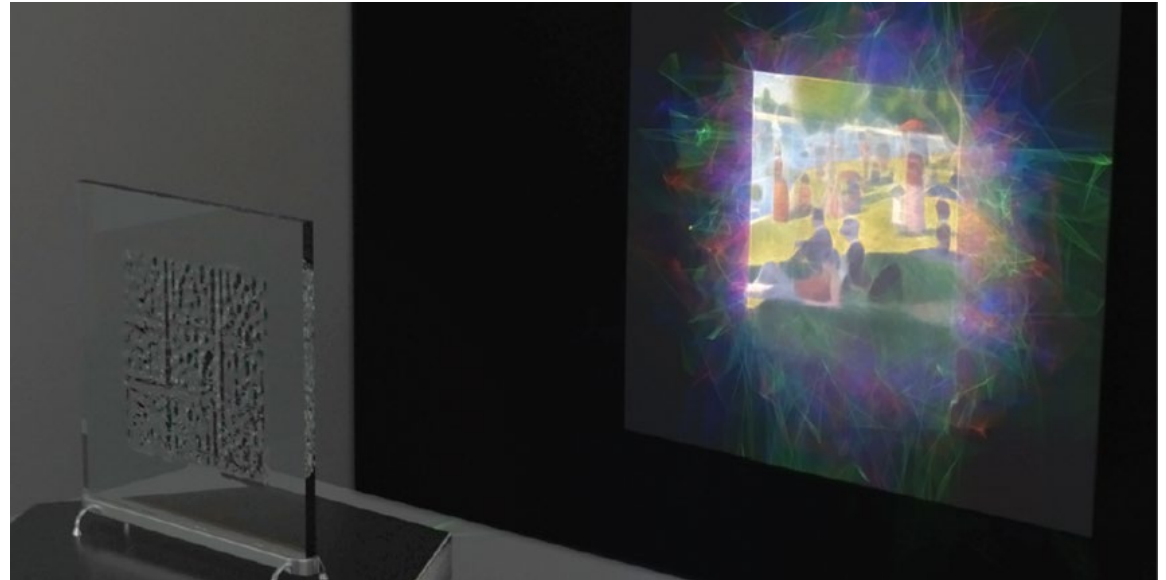
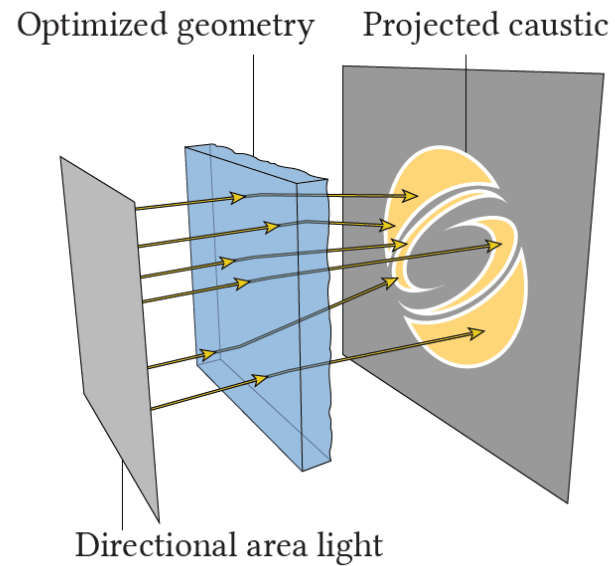
Optimized

Target (photo)

Figure from [\[Loubet et al. 2019\]](#)

Why differentiable rendering?

Caustic design



Images from [\[Nimier-David et al 2019\]](#)

Why differentiable rendering?

Color
optimization
for 3D printing

Naïve



Optimized



Reference

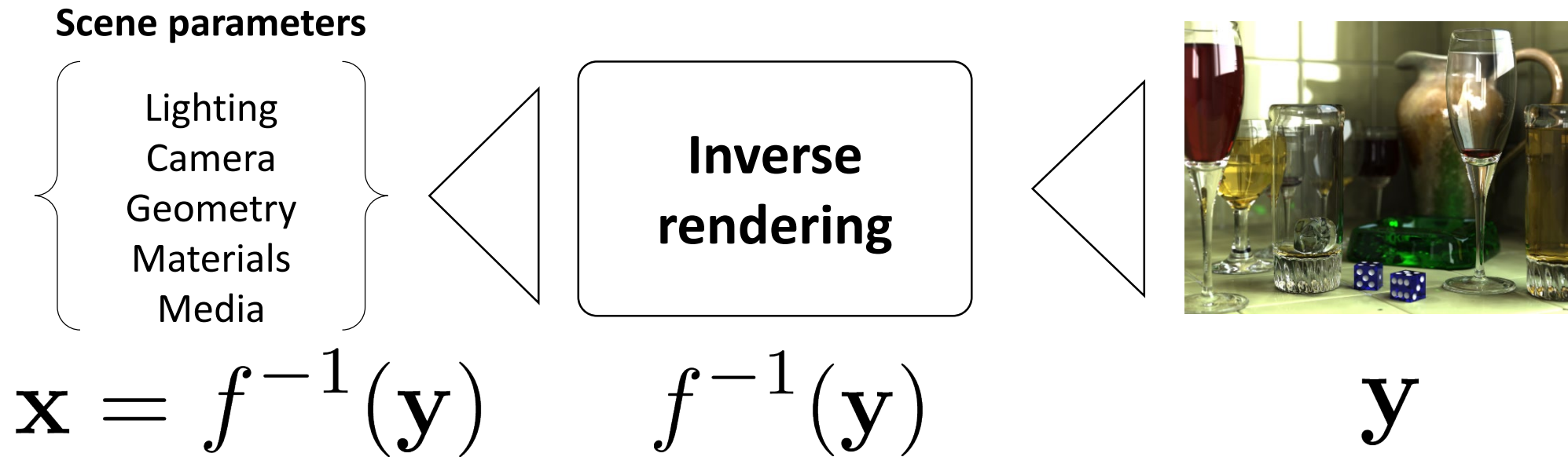


Images from [\[Nimier-David et al 2019\]](#)

Why differentiable rendering?

- **Inverse rendering:**

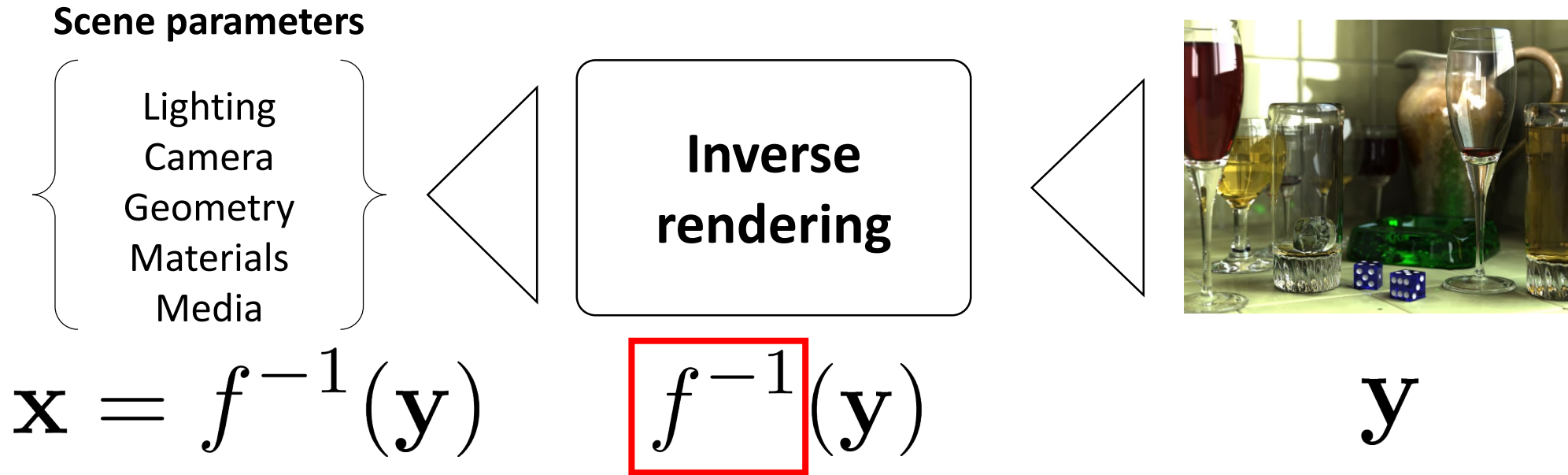
Given an image, compute the corresponding scene parameters



Why differentiable rendering?

- **Inverse rendering:**

Given an image, compute the corresponding scene parameters

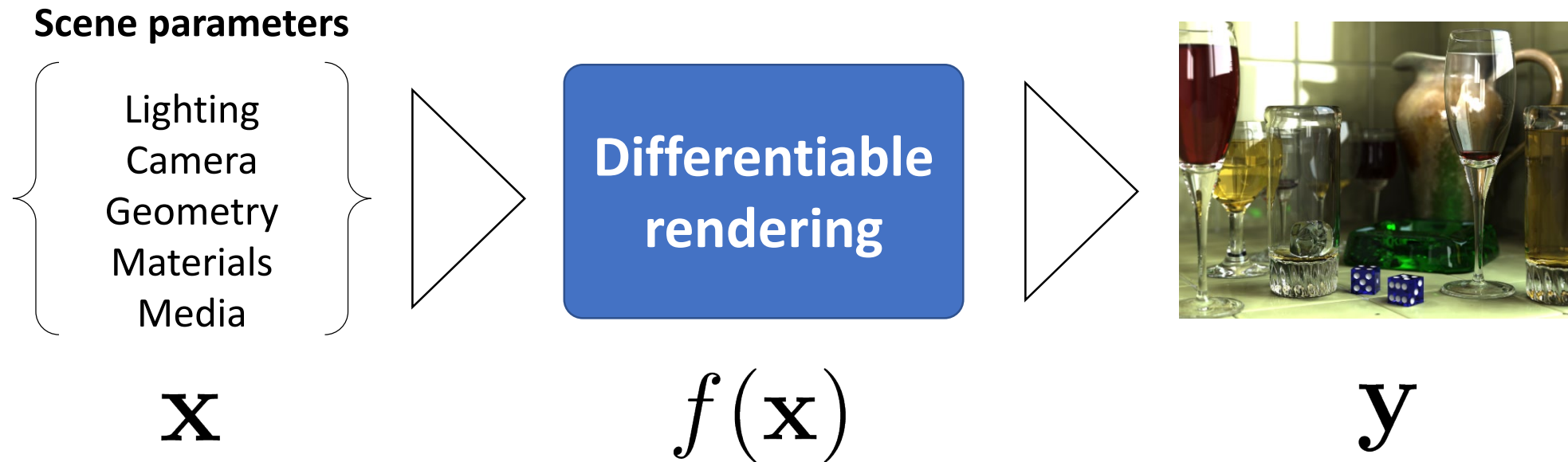


Too complex
 f likely not invertible

Why differentiable rendering?

- Differentiable rendering:

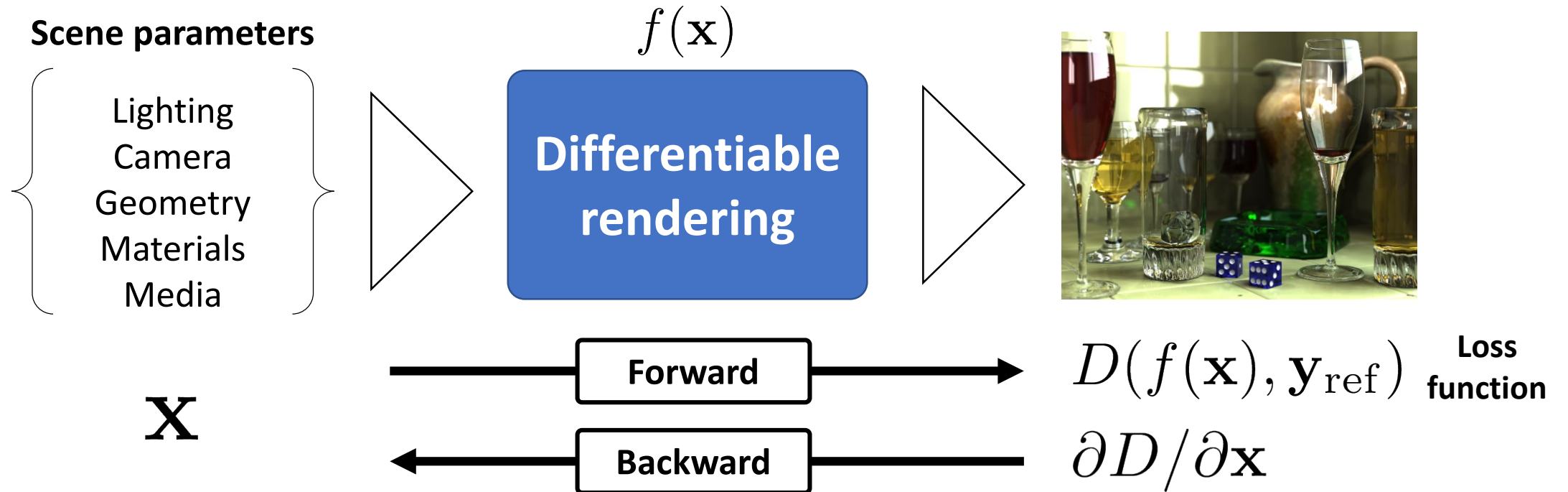
Given an image, **compute scene parameters** through **gradient-based optimization**



Why differentiable rendering?

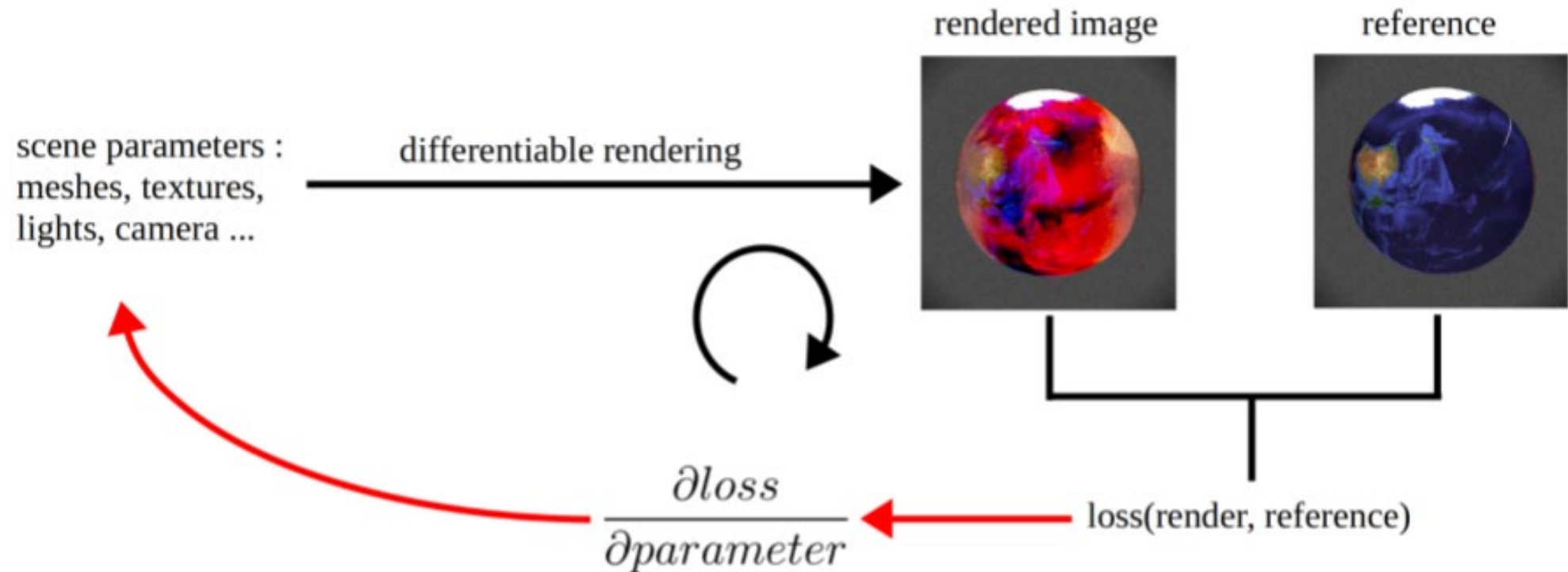
- Differentiable rendering:

Given an image, **compute scene parameters** through **gradient-based optimization**



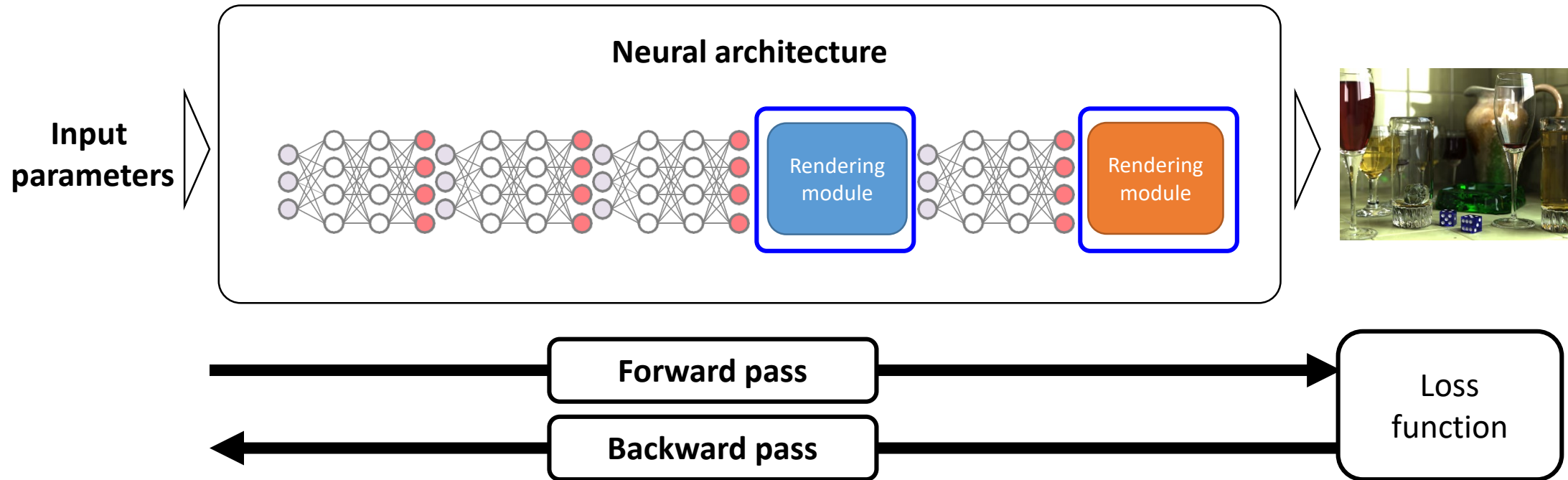
Why differentiable rendering?

- Differentiable rendering:
Given an image, **compute scene parameters** through **gradient-based optimization**



[Source](#)

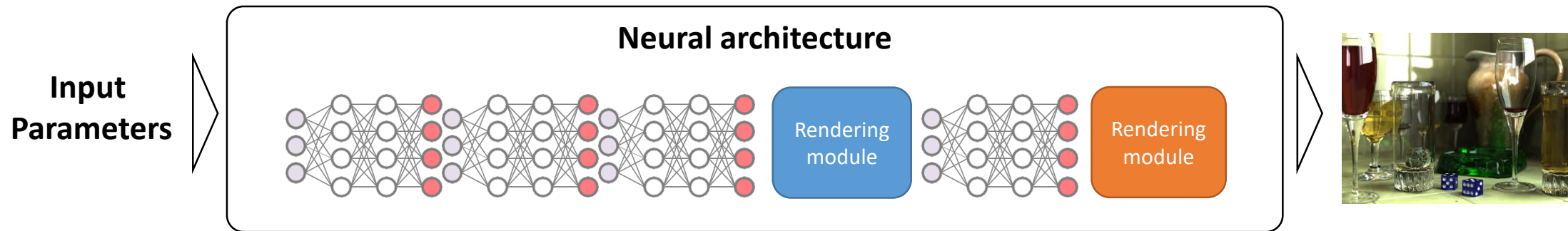
Previous lecture: Neural rendering



Differentiable vs neural rendering

Neural rendering:

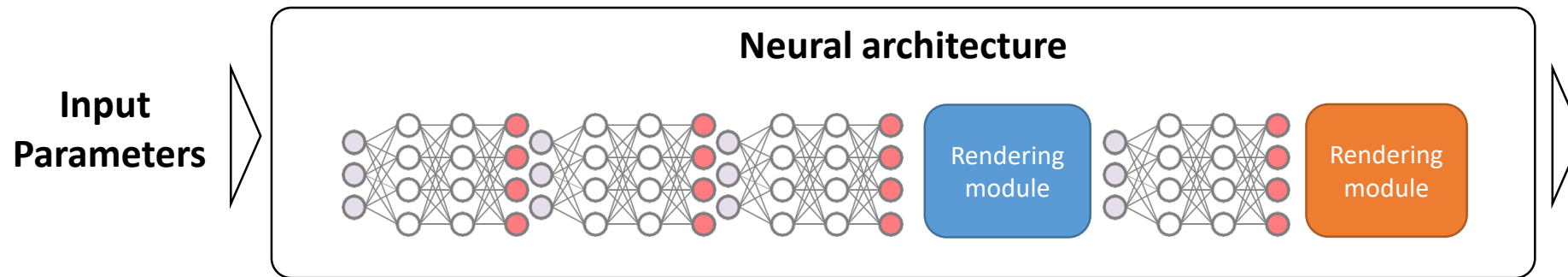
➔ Few input parameters (task-driven), neural representations



Differentiable vs neural rendering

Neural rendering:

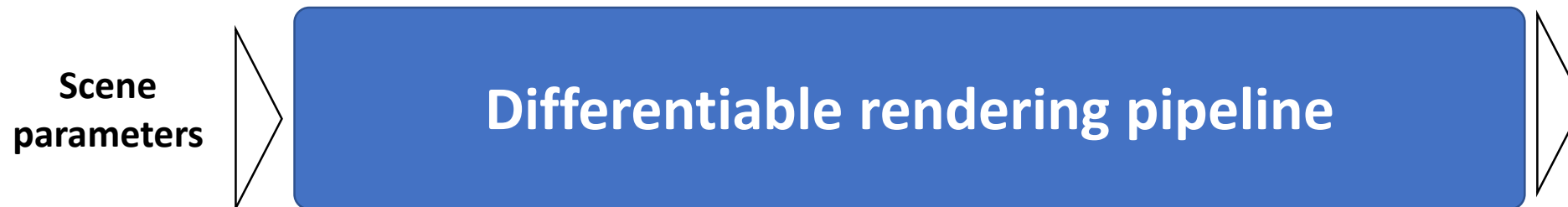
➔ Few input parameters (task-driven), **neural representations**



Differentiable rendering:

➔ No neural modules, **full differentiable rendering pipeline**

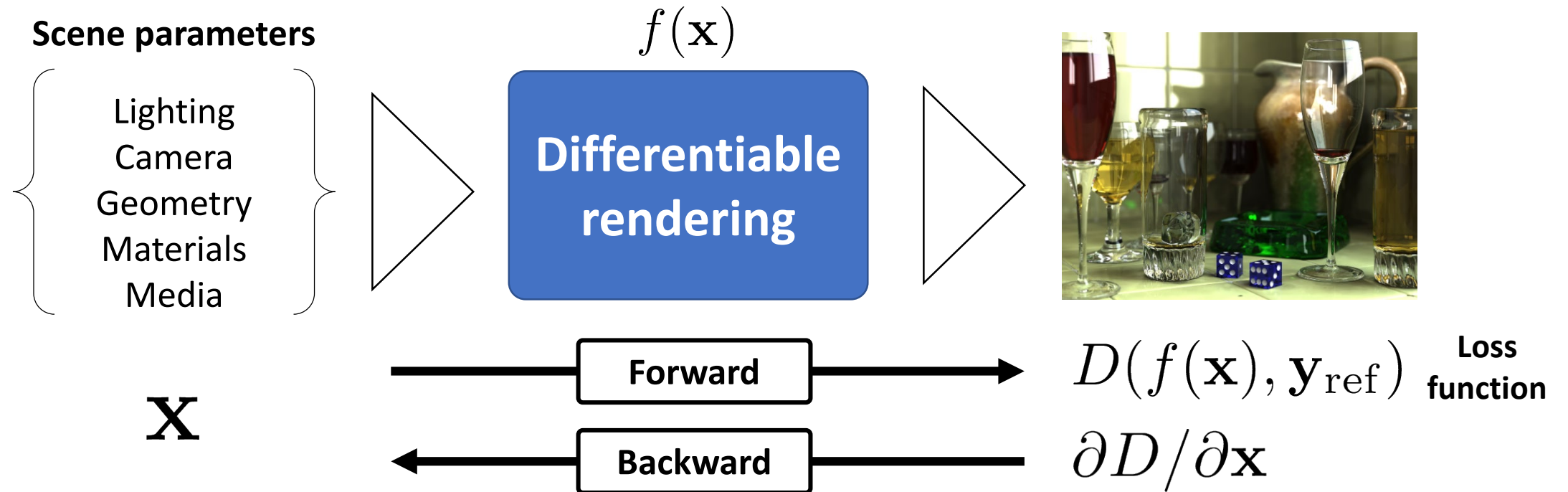
➔ **All scene parameters as input**, no neural simplification



Why differentiable rendering?

- Differentiable rendering:

Given an image, **compute scene parameters** through **gradient-based optimization**



More on this: [\[Kato et al.\]](#) Differentiable Rendering: A Survey

Example: Redner

Redner [\[Li et al. 2018\]](#):

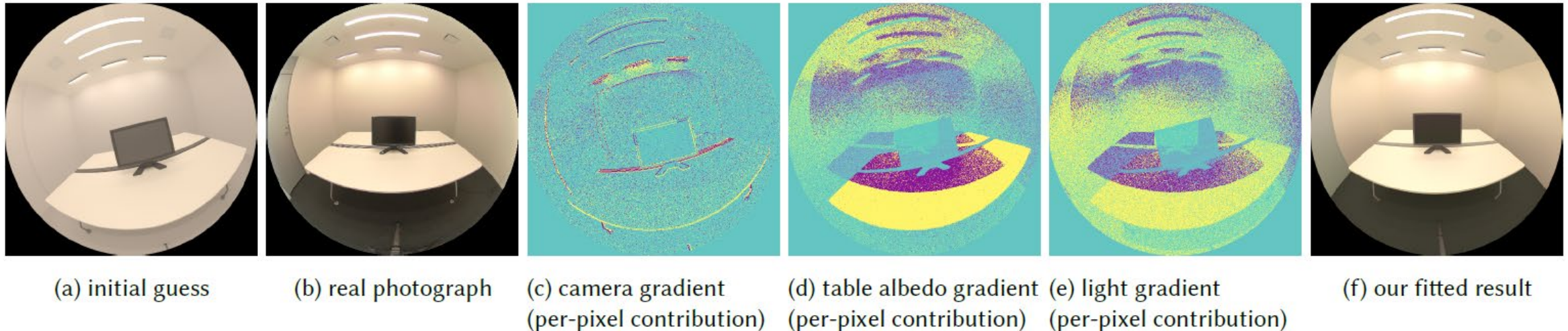
- First comprehensive Monte Carlo differentiable renderer
- Optimization of viewpoint, geometry, materials, lighting
- Unbiased, support for discontinuities

Example: Redner

Redner [\[Li et al. 2018\]](#):

- First comprehensive Monte Carlo differentiable renderer
- Optimization of viewpoint, geometry, materials, lighting
- Unbiased, support for discontinuities

Figure from [\[Li et al. 2018\]](#)

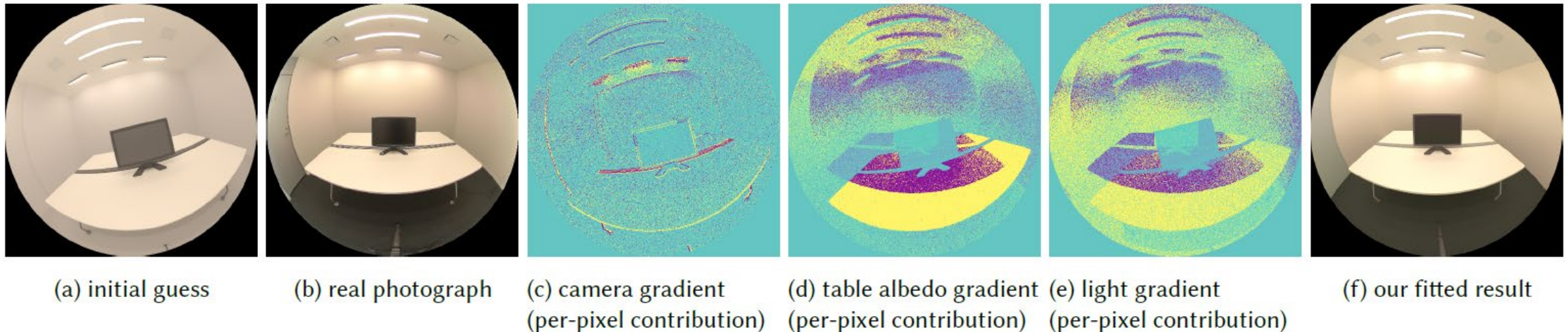


Example: Redner

Redner [\[Li et al. 2018\]](#):

- First comprehensive Monte Carlo differentiable renderer
- Optimization of viewpoint, geometry, materials, lighting
- Unbiased, support for discontinuities
- **Manual differentiation of relevant expressions**

Figure from [\[Li et al. 2018\]](#)



Example: Mitsuba 2

Mitsuba 2 [\[Nimier-David et al. 2019\]](#):

- Generic, physically-based differentiable rendering framework
- Automatic differentiation, computation graphs
- Research-oriented, retargetable, modular

Example: Mitsuba 2

Mitsuba 2 [\[Nimier-David et al. 2019\]](#):

- Generic, physically-based differentiable rendering framework
- Automatic differentiation, computation graphs
- Research-oriented, retargetable, modular

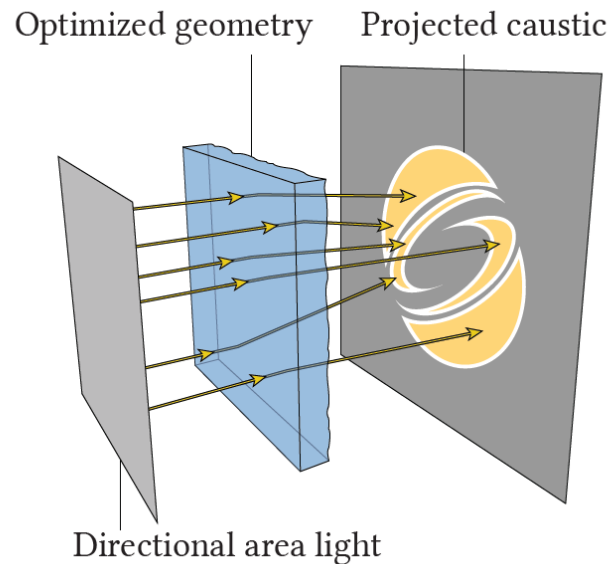


Images from [\[Nimier-David et al 2019\]](#)

Example: Mitsuba 2

Mitsuba 2 [\[Nimier-David et al. 2019\]](#):

- Generic, physically-based differentiable rendering framework
- Automatic differentiation, computation graphs
- Research-oriented, retargetable, modular



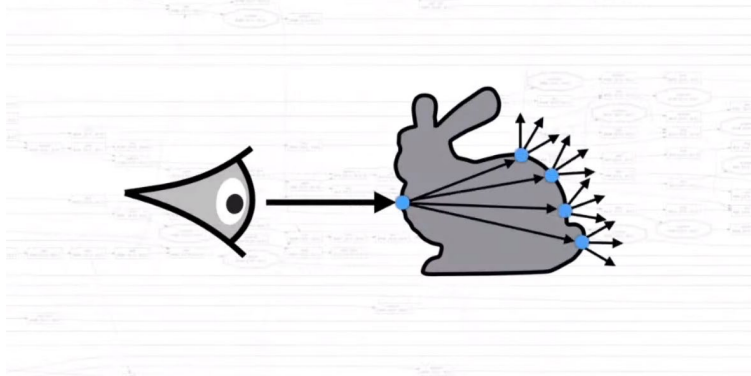
Images from [\[Nimier-David et al 2019\]](#)

Example: Mitsuba 2

Mitsuba 2 [\[Nimier-David et al. 2019\]](#):

- Generic, physically-based differentiable rendering framework
- Automatic differentiation, computation graphs
- Research-oriented, retargetable, modular

Light scattering, unstructured

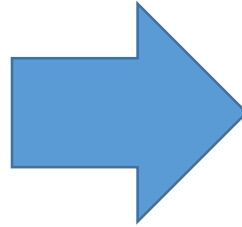
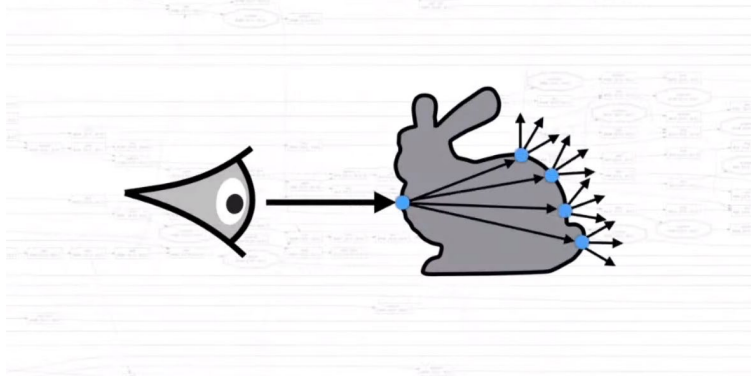


Example: Mitsuba 2

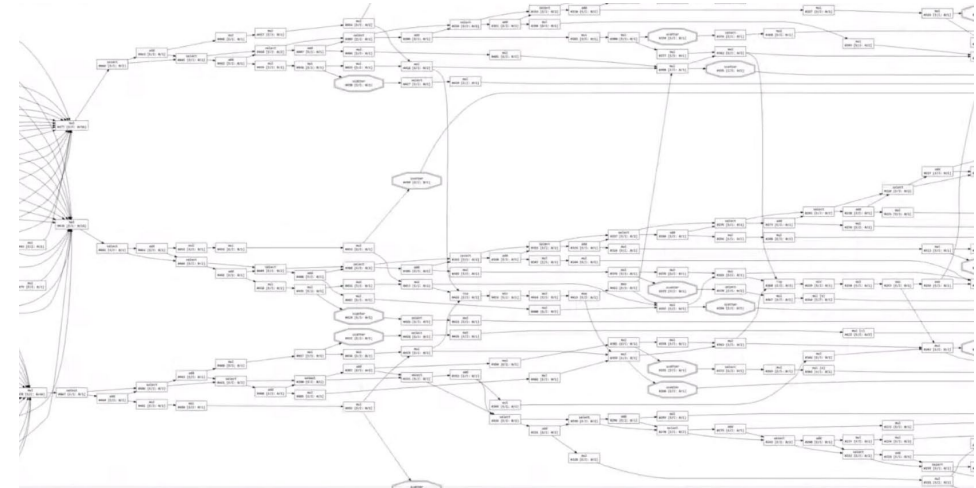
Mitsuba 2 [\[Nimier-David et al. 2019\]](#):

- Generic, physically-based differentiable rendering framework
- Automatic differentiation, computation graphs
- Research-oriented, retargetable, modular

Light scattering, unstructured



Huge computation graphs

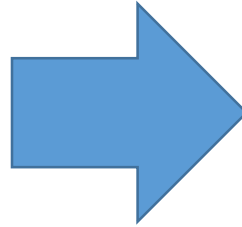
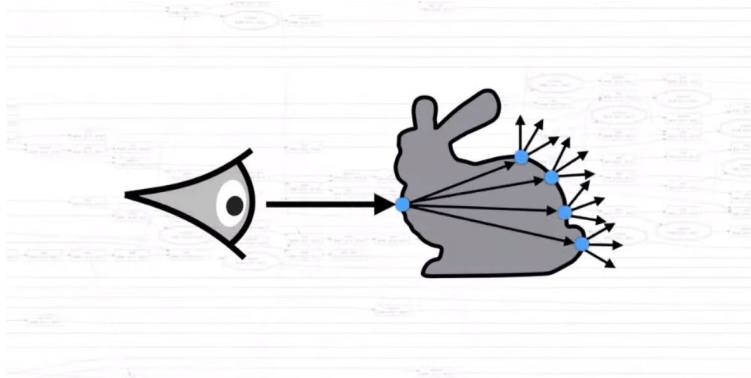


Example: Mitsuba 2

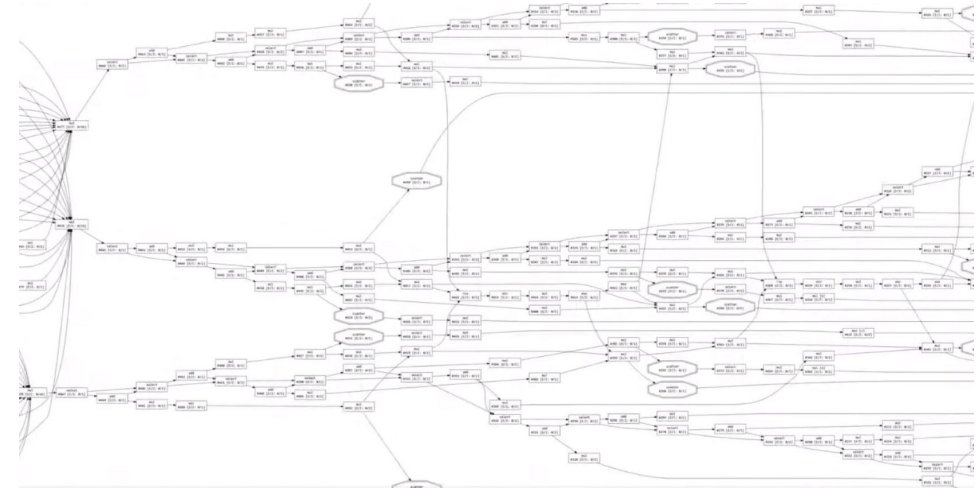
Mitsuba 2 [\[Nimier-David et al. 2019\]](#):

- Generic, physically-based differentiable rendering framework
- Automatic differentiation, computation graphs → **Memory constraints**
- Research-oriented, retargetable, modular

Light scattering, unstructured



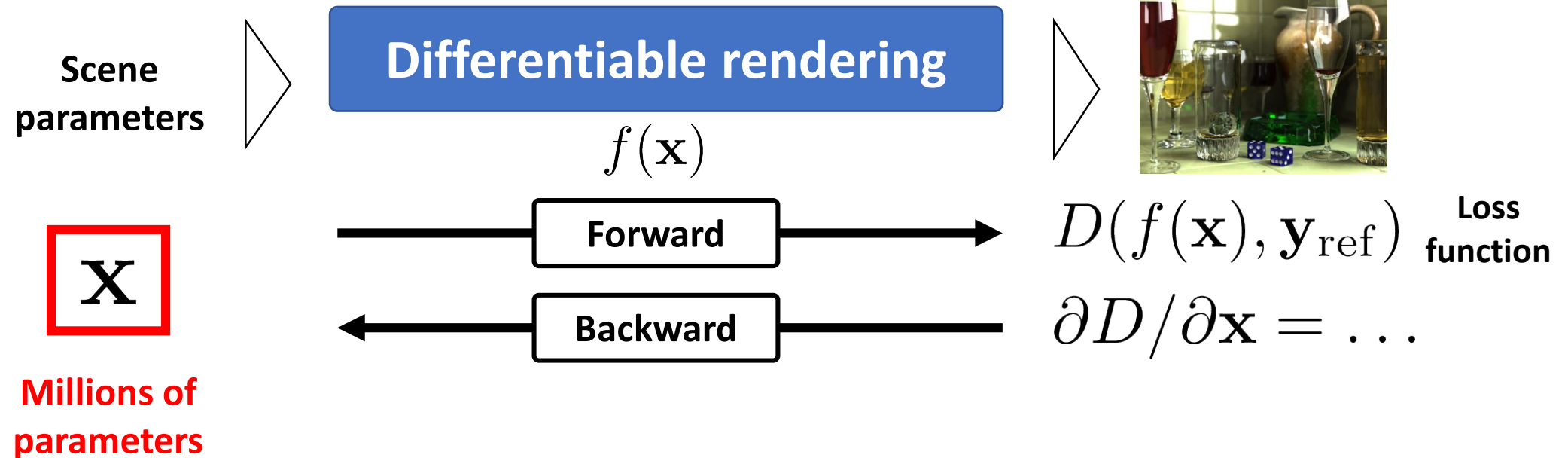
Huge computation graphs



Example: Mitsuba 2

Mitsuba 2 [\[Nimier-David et al. 2019\]](#):

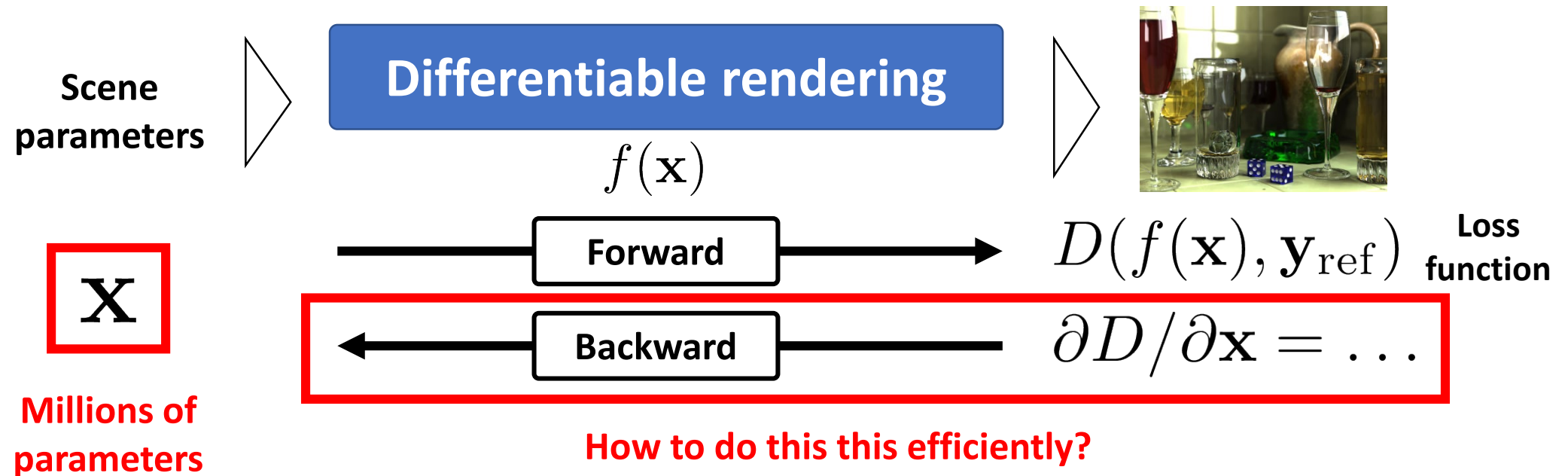
- Generic, physically-based differentiable rendering framework
- Automatic differentiation, computation graphs → **Memory constraints**
- Research-oriented, retargetable, modular



Example: Mitsuba 2

Mitsuba 2 [\[Nimier-David et al. 2019\]](#):

- Generic, physically-based differentiable rendering framework
- Automatic differentiation, computation graphs → **Memory constraints**
- Research-oriented, retargetable, modular



Example: Radiative backpropagation

Radiative backpropagation [\[Nimier-David et al. 2020\]](#):

- Path-trace radiance gradients during backpropagation
- Lifts memory constraints, full computation graph not stored

Example: Radiative backpropagation

Radiative backpropagation [\[Nimier-David et al. 2020\]](#):

- Path-trace radiance gradients during backpropagation
- Lifts memory constraints, full computation graph not stored

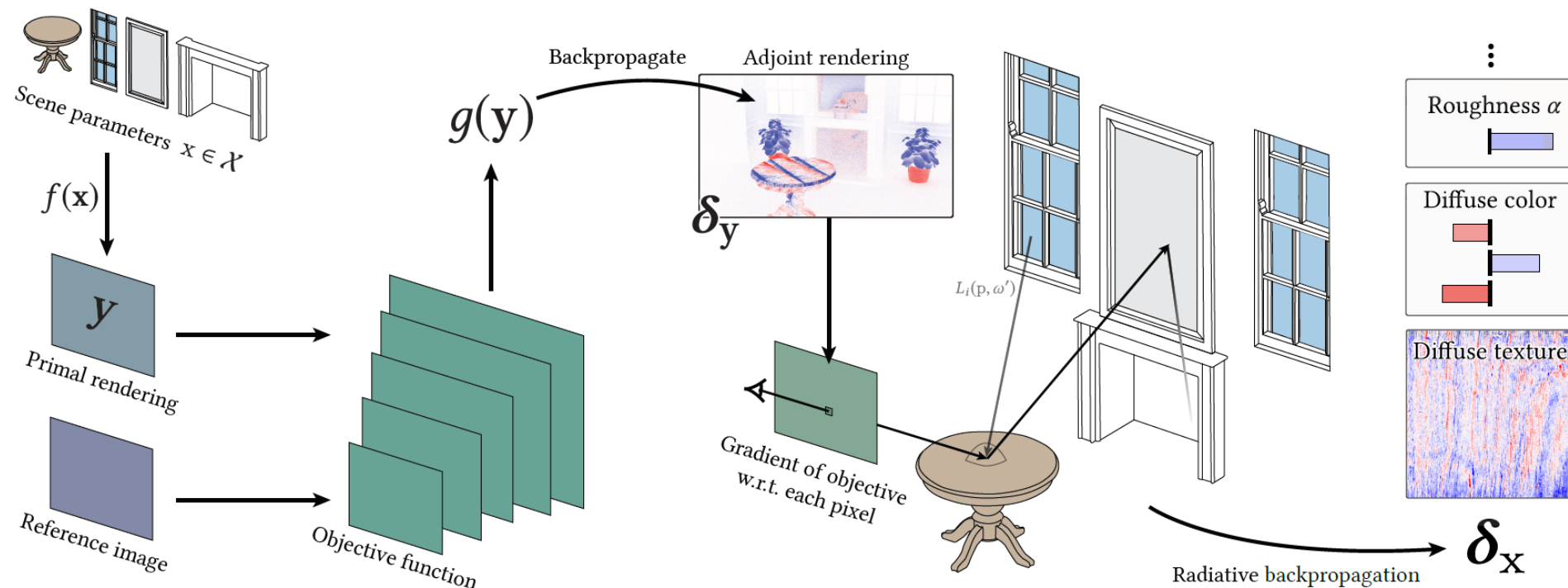


Figure from [\[Nimier-David et al. 2020\]](#)

Example: Radiative backpropagation

Radiative backpropagation [\[Nimier-David et al. 2020\]](#):

- Path-trace radiance gradients during backpropagation
- Lifts memory constraints, full computation graph not stored

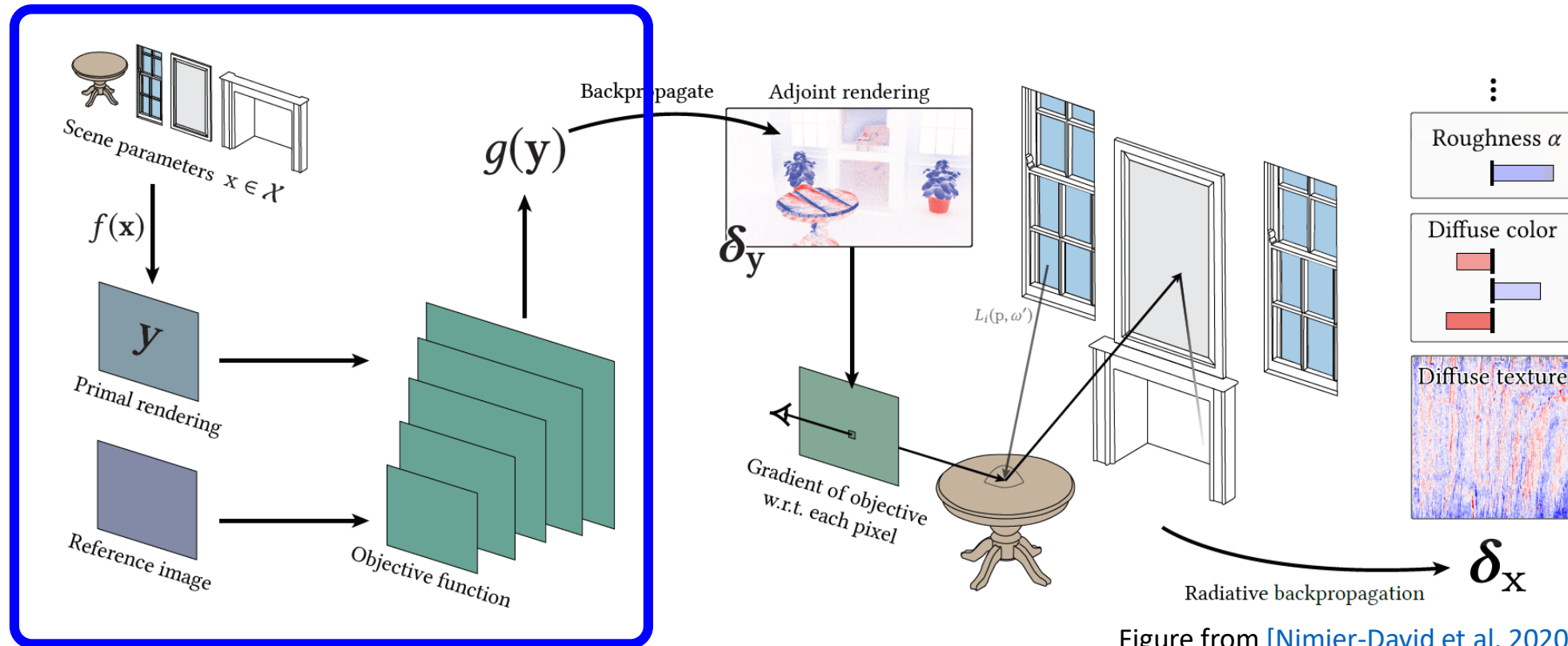


Figure from [\[Nimier-David et al. 2020\]](#)

Example: Radiative backpropagation

Radiative backpropagation [\[Nimier-David et al. 2020\]](#):

- Path-trace radiance gradients during backpropagation
- Lifts memory constraints, full computation graph not stored

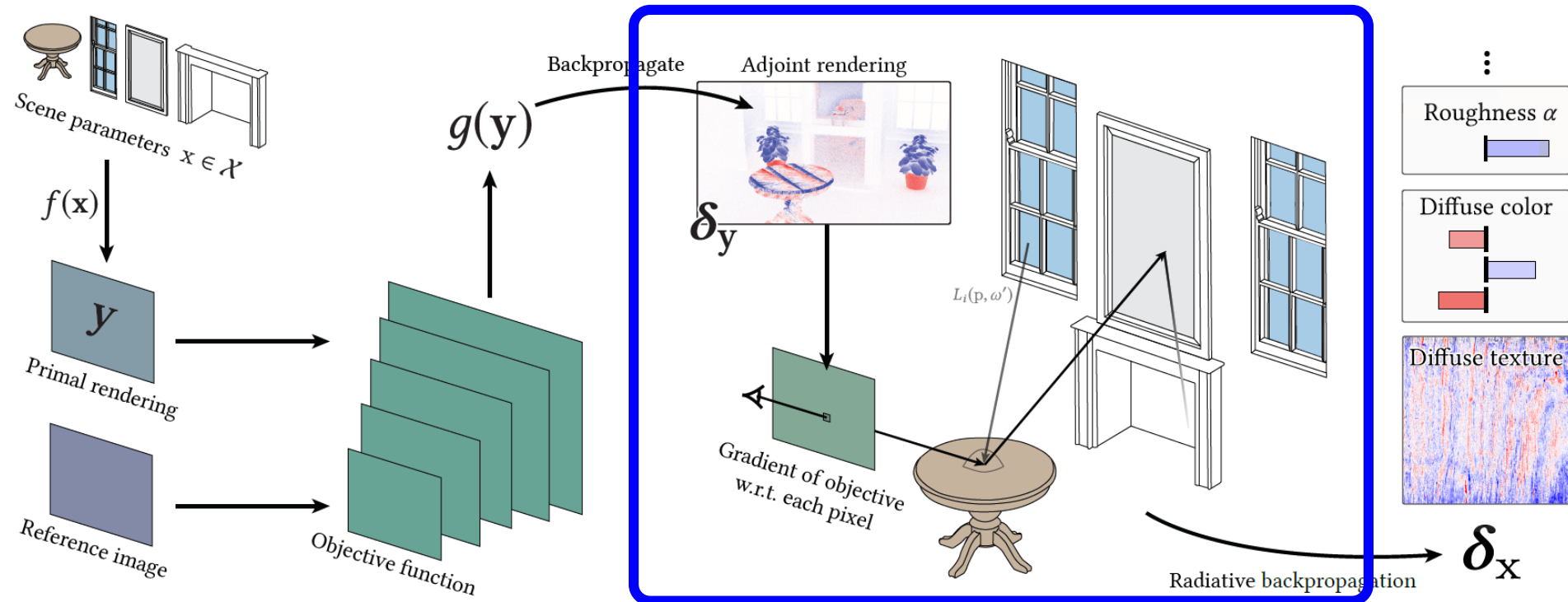


Figure from [\[Nimier-David et al. 2020\]](#)

Example: Radiative backpropagation

Radiative backpropagation [\[Nimier-David et al. 2020\]](#):

- Path-trace radiance gradients during backpropagation
- Lifts memory constraints, full computation graph not stored

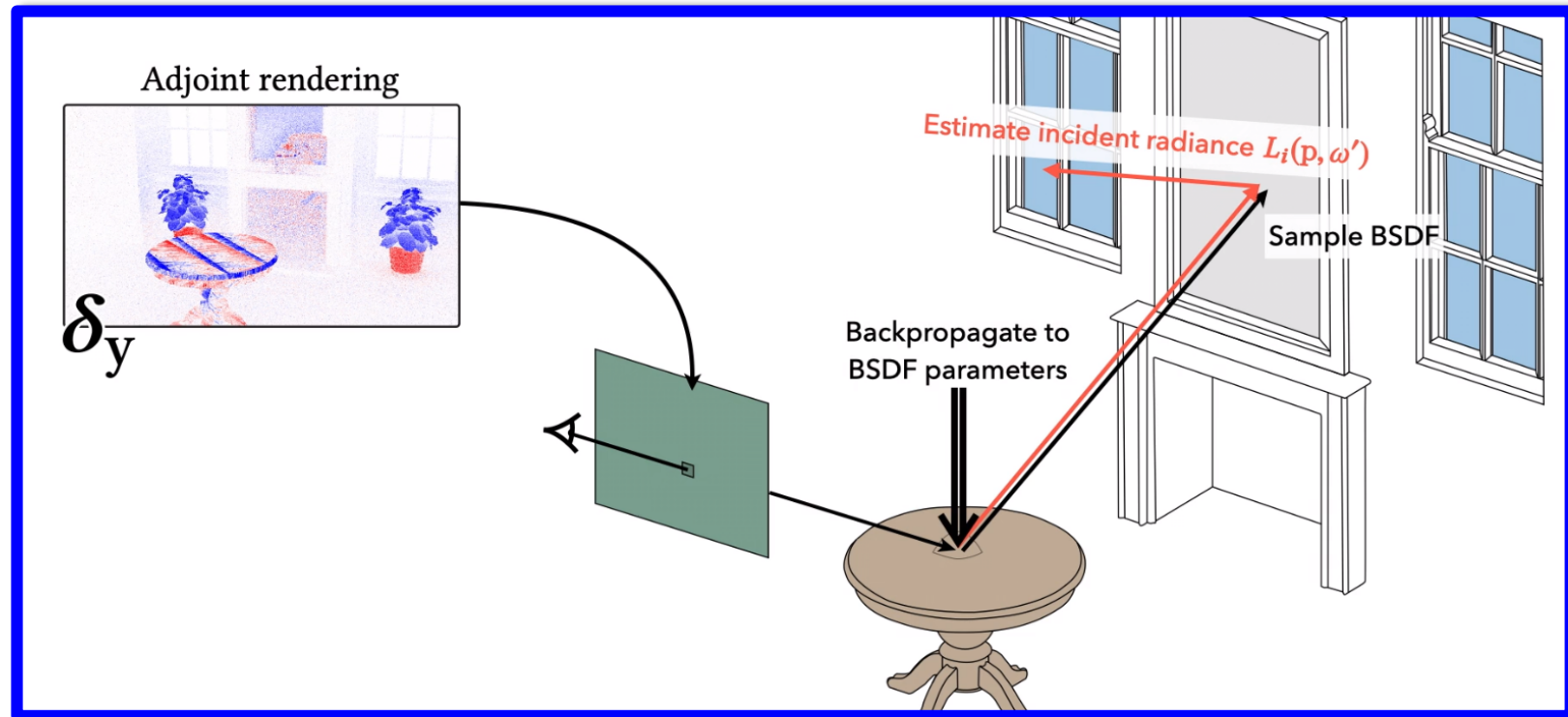


Figure from [\[Nimier-David et al. 2020\]](#)

Example: Radiative backpropagation

Radiative backpropagation [\[Nimier-David et al. 2020\]](#):

- Path-trace radiance gradients during backpropagation
- Lifts memory constraints, full computation graph not stored
- Up-to 1000x speed-ups w.r.t. Mitsuba 2's autodifferentiation

Example: Radiative backpropagation

Radiative backpropagation [\[Nimier-David et al. 2020\]](#):

- Path-trace radiance gradients during backpropagation
- Lifts memory constraints, full computation graph not stored
- Up-to 1000x speed-ups w.r.t. Mitsuba 2's autodifferentiation

Source



Target



Optimized



Example: Radiative backpropagation

Radiative backpropagation [\[Nimier-David et al. 2020\]](#):

- Path-trace radiance gradients during backpropagation
- Lifts memory constraints, full computation graph not stored
- Up-to 1000x speed-ups w.r.t. Mitsuba 2's autodifferentiation

Source



Target



Mitsuba 2



Radiative BP

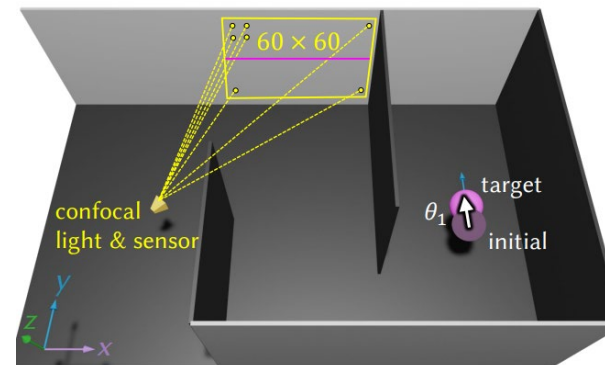
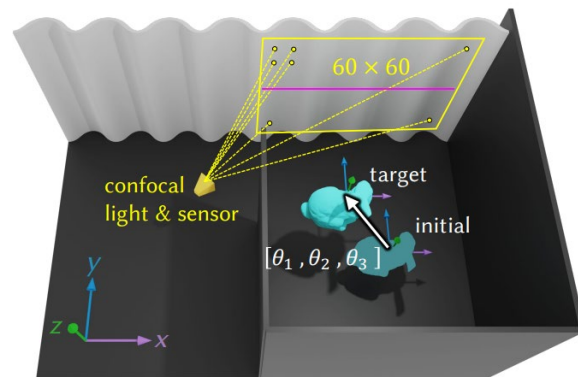


2.5 minutes

Example: Differentiable transient rendering

Differentiable transient rendering [\[Yi et al. 2022\]](#):

- Derivatives of time-resolved light transport in complex scenes.
- Enables inverting illumination in the temporal domain.
- More information than in regular steady-state transport.
- NLOS scene understanding:
 - Single-corner hidden objects 3D position estimation.
 - *Two*-corner hidden objects 1D position estimation.



Example: Mitsuba 3

- Major re-design of Mitsuba 2
- Allows developing rendering code in Python
- Just-in-time compilation of C++ megakernels for CPU and GPU
- Memory efficient in both CPU and GPU
- <https://www.mitsuba-renderer.org/>